



VELS





INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

An Intelligent Crowd Density Monitoring and Alert System for Safe Public Gatherings

FINAL YEAR PROJECT

PHASE-I REPORT

Submitted by

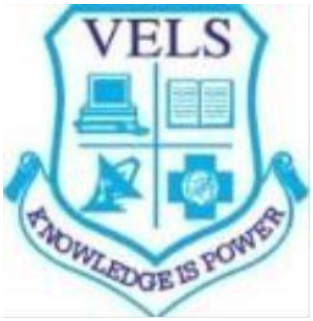
S.Udhayapradap (23619142)

V.Vigneshwaran(23619142)

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project "An Intelligent Crowd Density Monitoring and Alert System for Safe Public Gatherings" is the bonafide work of **S.Udhayapradap (23619142), V.Vigneshwaran(23619142)** who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD

Dr. K. Ulaga Priya, M.E., Ph.D
Associate Professor & Head
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

SIGNATURE OF THE SUPERVISOR

Mrs S.M. Sowmiya, M.E., (Ph.D.)
Assistant Professor
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600117.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities have been a motivating factor in this completion of project work, all because of FOUNDER & CHAIRMAN, Shri. Dr.ISHARIK. GANESH., with great pleasure that I take this opportunity to thank him. I am grateful to our HEAD OF THE DEPARTMENT, Dr. K. Ulagapriya., for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. I extend our warmest thanks to my INTERNAL GUIDE, Mrs. S.M. Sowmiya, who has patiently guided and provided us with invaluable advice and help. I am thankful to my Project Coordinator Ms. S. J. Kavitha. I would like to express my greatest gratitude to our Staff members, Librarian, and Non-teaching staff members of Computer Centre, who have helped and supported me throughout. Last but not the Least, I wish to thank my parents and my friends for their individual support and interest who has inspired me and encouraged me to go in my own way, without them I would be unable to complete this project.

S.UDHAYAPRADAP 23619140
V.VIGNESHWARAN 23619142

TABLE OF CONTENTS

LIST OF TABLES ..... **iii**
LIST OF FIGURES ..... **iv**
LIST OF ABBREVIATIONS ..... **v**
CHAPTER 1 INTRODUCTION **1**
1.1 INTRODUCTION 1
1.2 CHALLENGES IN TEMPLE CROWD MANAGEMENT 3
1.3 PROBLEM STATEMENT 5
1.4 MOTIVATION 6
1.5 RESEARCH OBJECTIVES 8
1.7 ORGANIZATION OF THE REPORT 11
CHAPTER 2 LITERATURE REVIEW **12**
2.1 INTRODUCTION 12
2.3 AI-BASED CROWD MONITORING SYSTEMS 14
2.4 IOT-BASED CROWD MONITORING SYSTEMS 20
2.6 RESEARCH GAPS IN EXISTING CROWD MONITORING SYSTEMS 25
2.7 SUMMARY OF LITERATURE REVIEW 29
CHAPTER 3 SYSTEM ANALYSIS AND DESIGN **31**
3.1 INTRODUCTION 31
3.2 EXISTING SYSTEM AND ITS LIMITATIONS 34
3.3 PROPOSED SYSTEM 37
3.4 SYSTEM ARCHITECTURE DIAGRAM 44
3.5 HARDWARE REQUIREMENTS 47
3.6 SOFTWARE REQUIREMENTS 51
CHAPTER 4 RESULTS AND DISCUSSION **60**
4.1 INTRODUCTION 60
4.2 EXPERIMENTAL SETUP 61
4.3 FUNCTIONAL DEMONSTRATION RESULTS 61
4.4 PERSON DETECTION AND TRACKING RESULTS 62
4.5 CROWD DENSITY CALCULATION RESULTS 63
4.6 SENSOR VALIDATION RESULTS 63
4.7 PROCESSING SPEED ANALYSIS 64
4.8 CROWD GROWTH AND TREND MONITORING RESULTS 64
4.9 ALERT GENERATION PERFORMANCE 65
4.10 OVERALL RESULTS AND DISCUSSION 65
4.11 CHAPTER SUMMARY 66
CHAPTER 5 CONCLUSION AND FUTURE ENHANCEMENTS **67**
5.1 INTRODUCTION 67
5.2 CONCLUSION 67
5.3 MAJOR CONTRIBUTIONS OF THE PROJECT 68
5.4 LIMITATIONS OF THE PROJECT 69
5.5 FUTURE ENHANCEMENTS 69
5.6 APPLICATIONS OF CROWDGUARD AI 71
5.7 FINAL CONCLUSION 71
REFERENCES **72**
APPENDIX A SOURCE CODE **75**
A.1 IMPORTS, ESP32 SERIAL LINK AND MODEL LOADING 75
A.2 CAMERA INPUT, CONSTANTS AND ALARM FUNCTION 76
A.3 MAIN LOOP: SERIAL INPUT AND FRAME CAPTURE 76
APPENDIX B SCREENSHOT 78

Abstract

Large public events such as temple festivals and religious processions in Tamil Nadu often go beyond the crowd-control capacity of narrow passages, entrances, and staircases, leading to dangerous stampedes. Currently, crowd management depends mainly on manual observation and a afterwards review of CCTV footage, a method that is unable to reliably identify an increasing danger before it becomes obvious. The project, An Intelligent Crowd Density Monitoring and Alert System for Safe Public Gatherings, suggests a low-cost system based on IoT and AI that continuously monitors high-risk chokepoints by using both a camera and an IR/laser break-beam sensor. The computer vision model based on the camera estimates the real-time crowd density in a given area, while the laser beam sensor separately counts the number of people passing through that point, enabling the system to cross-check its readings even in cases of poor lighting or when the camera is obstructed. Instead of merely responding to the current crowd level, the system monitors the rate at which density is increasing in each area and compares this rate with a safe occupancy limit that is specific to the physical width of that location. If a zone is approaching its danger level, an automatic alert is sent to the event organisers and to the staff in the control room, giving an early warning before the situation becomes critical. The system is constructed using inexpensive and easily obtained hardware, so that it can be put into use by temple trusts and local authorities at various chokepoints rather than depending on a single central monitoring post. The proposed system wants to close the gap between research on crowd monitoring and the real-world situations, in India where public gatherings happen often with limited resources. It does this through the use of sensor fusion setting thresholds which are particular to every zone, as well as predictions based upon trends.

Keywords— IoT, Crowd Density Estimation, Computer Vision, Sensor Fusion, IR/Laser Break-Beam Sensor, Stampede Prediction, Real-Time Alert System, Chokepoint Monitoring, Machine Learning, Smart Surveillance

LIST OF TABLES

Table No.	Title	Page
Table 2.1	Comparison of Crowd Monitoring Methods	14
Table 2.2	Role of Each Component in CrowdGuard AI	18
Table 2.3	Hardware Components and Their Functions	24
Table 2.4	Research Gaps and Proposed Approach	29
Table 2.5	Key Findings of the Literature Review	30
Table 3.1	System Layers of CrowdGuard AI	32
Table 3.2	Comparison of Existing and Proposed Systems	36
Table 3.3	Data Flow Between System Components	46
Table 3.4	List of Hardware Components	48
Table 3.5	Hardware Requirements Summary	51
Table 3.6	List of Software Requirements	52
Table 3.7	Functional Requirements	56
Table 3.8	Non-Functional Requirements	57
Table 3.9	Use Cases of the System	57
Table 3.10	Threshold Decision Examples	57
Table 3.11	Warning Levels	57
Table 3.12	Technology Summary	58
Table 3.13	Unit Test Cases	58
Table 3.14	Camera-Based and IR/Laser Sensor Monitoring Compared	59
Table 3.15	Threshold Test Conditions	59
Table 3.16	Performance Evaluation Methods	59
Table 3.17	Limitations of the Proposed System	60
Table 4.1	Hardware Setup	61
Table 4.2	Software Setup	61
Table 4.3	Functional Demonstration Results	62

Table No.	Title	Page
Table 4.4	Person Detection Evaluation Metrics	62
Table 4.5	Sensor Validation Results	63
Table 4.6	Processing Speed Observations	64
Table 4.7	Alert Response Time by Configuration	65
Table 5.1	Application Areas of CrowdGuard AI	71

LIST OF FIGURES

Figure No.	Title	Page
Figure 1.1	Overall Workflow of CrowdGuard AI	3
Figure 2.1	Camera and Sensor Processing Flow of CrowdGuard AI	14
Figure 3.1	System Architecture of CrowdGuard AI	45

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
CAM	Camera (as in ESP32-CAM)
CCTV	Closed-Circuit Television
CNN	Convolutional Neural Network
COM	Communication Port (serial port)
FPS	Frames Per Second
ID	Identifier
IDE	Integrated Development Environment
IoT	Internet of Things
IR	Infrared
LED	Light Emitting Diode
MAE	Mean Absolute Error
OpenCV	Open Source Computer Vision Library
RFID	Radio-Frequency Identification
ROI	Region of Interest
USB	Universal Serial Bus
UT	Unit Test
YOLO	You Only Look Once
YOLOv8n	You Only Look Once, version 8 (nano variant)

CHAPTER 1

INTRODUCTION

1.1 Introduction

Public safety has become an important concern in places where large numbers of people gather, such as temples, festivals, railway stations, bus terminals, shopping malls, stadiums, and other public spaces [6]. During religious festivals and public celebrations, thousands of people may gather in a limited area within a short period [6]. When the number of people increases beyond the safe capacity of a location, overcrowding can occur, creating difficulties in movement and increasing the risk of accidents and stampedes [8]. Therefore, an effective crowd monitoring system is necessary to observe crowd movement, estimate crowd density, identify potentially dangerous situations, and generate timely alerts [8], [20].

Traditional crowd management systems mainly depend on security personnel, manual observation, and CCTV surveillance [6], [21]. Although these methods are useful, they have certain limitations. Security personnel may find it difficult to continuously monitor multiple locations simultaneously [6]. Similarly, conventional CCTV systems generally provide video footage, but identifying overcrowding and predicting dangerous situations still requires continuous attention from human operators [21]. When crowd density increases rapidly, manual observation may not provide sufficient time to respond. These limitations highlight the need for an intelligent system that can automatically monitor crowd conditions and assist event organisers in making timely safety decisions.

To address these challenges, the project "CrowdGuard AI: An Intelligent Crowd Density Monitoring and Alert System for Safe Public Gatherings" proposes an integrated crowd monitoring solution that combines Artificial Intelligence (AI), computer vision, and Internet of Things (IoT) technologies [1], [19]. The primary purpose of this system is to continuously monitor a designated area, detect people, estimate the number of individuals present, evaluate crowd density, and generate alerts when predefined safety conditions are reached [19].

The proposed system uses the YOLOv8n object detection model to identify people in video frames [1], [14]. The camera captures video, which is processed using computer vision techniques. The detected individuals are then tracked across consecutive frames using the ByteTrack multi-object tracking algorithm [2]. Tracking helps maintain individual identities across frames and reduces the possibility of repeatedly counting the same person. By defining a specific monitoring region, the system can focus on a particular location, such as a narrow passage, entrance, or queue area, instead of analysing the entire camera view without considering spatial boundaries.

In addition to camera-based monitoring, CrowdGuard AI incorporates an IR/laser break-beam sensor connected to an ESP32 microcontroller [20], [21]. When a person crosses the monitored beam, the sensor produces an event that can be recorded by the controller [20], [21]. This provides a second source of information to complement the camera-based detection system. The camera estimates the number of people visible within a monitored

region, whereas the break-beam sensor records interruptions at a particular crossing point. These measurements provide different types of information and can be compared to identify possible differences between the camera observations and sensor events. The system does not simply add both counts together, because doing so could result in duplicate counting. Instead, the sensor readings support the monitoring and decision-making process.

Another important feature of CrowdGuard AI is its ability to evaluate crowd conditions using predefined thresholds and changes in crowd density over time. A threshold represents a configured limit for the number of people or the occupancy level within a monitored zone. When the crowd approaches the configured limit, the system can indicate a warning condition. If the critical condition is reached, the system activates a local alert using a buzzer and LED connected to the ESP32 controller. This allows nearby personnel to recognise a potentially unsafe situation and take appropriate action.

The system also considers the rate at which crowd density changes. A location may initially contain a manageable number of people, but a sudden increase in arrivals can cause congestion within a short period. Therefore, monitoring only the current crowd count may not provide sufficient information about the developing situation. By analysing changes in occupancy over time, CrowdGuard AI aims to identify rapidly increasing crowd levels and support early-warning decisions before the configured critical condition is reached. However, this feature should be understood as threshold-based risk estimation rather than a system that can guarantee the prediction of a future stampede.

The proposed architecture separates the computer vision processing tasks from the sensor monitoring and alert-control functions. A laptop or computer processes the video stream, performs person detection and tracking, calculates occupancy-related information, and displays the monitoring results. The ESP32 handles the break-beam sensor input and local alert devices. This arrangement allows the system components to operate as separate modules while contributing to a common monitoring process.

The overall workflow begins with video acquisition, followed by frame processing, person detection, person tracking, monitoring-zone identification, and people counting. The calculated count is evaluated against the configured threshold. If the monitored condition remains within the acceptable range, the system continues monitoring. If the configured critical condition is reached, an alert is generated. The monitoring cycle then continues so that changes in crowd conditions can be observed over time.

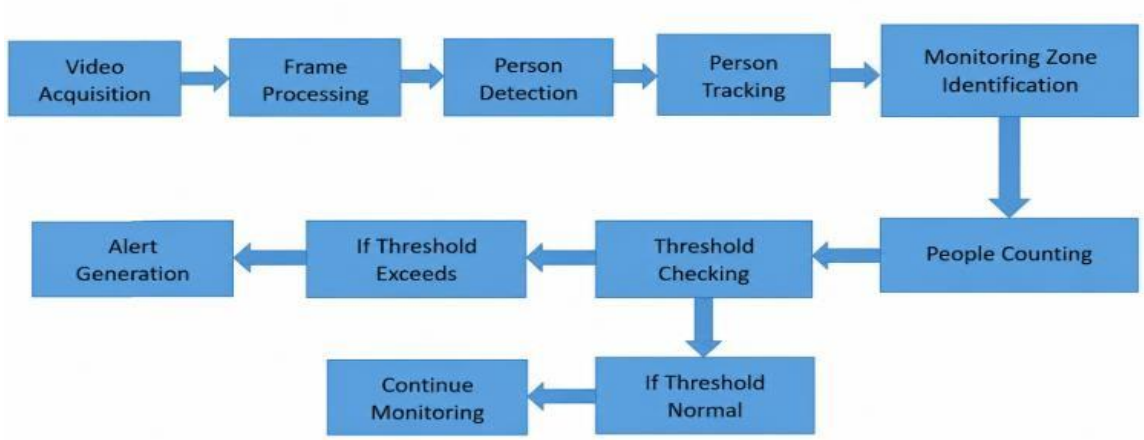


Figure 1.1: Overall Workflow of CrowdGuard AI

1.2 CHALLENGES IN TEMPLE CROWD MANAGEMENT

Managing crowds during temple festivals and religious gatherings is a challenging task because a large number of people may gather in a limited space within a short period [6]. In Tamil Nadu, temple festivals, religious processions, and other public celebrations often attract large crowds [6]. During these events, entrances, narrow passages, queue corridors, and staircases may become overcrowded [8]. Without proper monitoring, overcrowding can lead to restricted movement, congestion, and dangerous situations [8].

Traditional crowd management mainly depends on security personnel, CCTV cameras, barricades, and manual observation [6], [21]. Although these methods help maintain public safety, they have certain limitations when the number of people increases rapidly [6]. Crowd conditions can change within a few minutes, making continuous monitoring and timely decision-making essential.

The major challenges in temple crowd management are explained below.

1. Large Number of People in Limited Spaces

During temple festivals, a large number of visitors may enter through narrow entrances and queue corridors. When too many people occupy a small area, movement becomes difficult and congestion may develop. Monitoring the number of people in these locations is necessary to identify potentially unsafe conditions.

2. Limitations of Manual Monitoring

Security personnel must observe crowd movement, identify overcrowded areas, and coordinate with other staff members. When several locations need to be monitored simultaneously, it becomes difficult for personnel to maintain continuous attention to every area. Human observation may also delay the identification of rapidly changing crowd conditions.

3. Limitations of CCTV Surveillance

CCTV cameras provide continuous video footage of public areas [6], [22]. However, conventional surveillance systems generally depend on human operators to observe the footage and identify potential risks. When multiple camera feeds are displayed simultaneously, operators may find it difficult to recognise a sudden increase in crowd density in a particular location.

4. Occlusion and Visibility Problems

In crowded environments, people may partially or completely block one another from the camera's view [22], [29]. Pillars, temporary structures, banners, umbrellas, and other objects can also obstruct visibility [29]. Poor lighting and changes in camera angles may further affect the reliability of person detection [22]. These conditions can lead to missed detections or inaccurate people counting.

5. Difficulty in Monitoring Specific Zones

The total number of people in a large area does not always indicate the level of danger in a particular location. For example, an open courtyard may contain many people while a narrow entrance becomes dangerously congested. Therefore, crowd monitoring should consider individual zones and their available space rather than relying only on the overall crowd count.

6. Absence of Independent Sensor Verification

A camera-based system may experience difficulties when visibility is poor or people obstruct the camera [20]. An independent sensor can provide additional information about movement through a specific point [20]. However, a conventional break-beam sensor only records interruptions at its installation point and cannot independently determine the crowd distribution across a larger area. Combining camera observations with sensor events therefore requires suitable decision-making logic.

7. Delayed Warning Generation

Identifying an overcrowded area after it has already reached a critical condition may leave insufficient time for corrective action. Monitoring only the current number of people may also fail to indicate how quickly the crowd is increasing. Analysing occupancy changes over time can help identify developing congestion and support earlier warnings.

8. Difficulty in Establishing Safe Occupancy Limits

Different locations have different physical dimensions and crowd-handling requirements. A narrow passage cannot safely accommodate the same number of people as a large open area under otherwise comparable conditions. Therefore, the monitoring system should use appropriate thresholds for individual zones instead of applying one fixed limit to every location.

Role of CrowdGuard AI in Addressing These Challenges

CrowdGuard AI is designed to address these challenges by integrating artificial intelligence, computer vision, and IoT-based sensing into a unified monitoring framework [19]. The system uses YOLOv8n for person detection and ByteTrack for tracking detected individuals across consecutive video frames [1], [2]. It identifies a predefined monitoring zone and counts the people present within that region.

An IR/laser break-beam sensor connected to an ESP32 provides additional crossing-event information. The system evaluates camera-based observations, sensor events, crowd density, and configured occupancy thresholds to support crowd-risk assessment. When a critical condition is reached, a buzzer and LED provide a local warning.

The proposed system also considers changes in crowd conditions over time. This helps distinguish a relatively stable crowd from one whose occupancy is increasing rapidly. Although the approach supports early-warning decisions, its effectiveness in large public gatherings requires further testing under different crowd sizes and environmental conditions.

In conclusion, effective temple crowd management requires continuous observation, zone-specific occupancy monitoring, reliable people counting, and timely alert generation. CrowdGuard AI provides a prototype approach that combines these functions to support safer crowd management during religious festivals and other public gatherings.

1.3 PROBLEM STATEMENT

Crowd management during temple festivals, religious processions, and other public gatherings is a challenging task, particularly in areas where a large number of people move through narrow entrances, queue corridors, and restricted passages. Traditional crowd monitoring methods mainly depend on manual observation and CCTV surveillance. These methods may not provide sufficiently timely warnings when crowd density increases rapidly or congestion develops in a specific location.

Conventional CCTV systems provide visual information but generally require human operators to observe the footage and identify potentially dangerous situations [6], [22]. Monitoring multiple locations simultaneously can increase the workload of security personnel and make it difficult to recognise sudden changes in crowd conditions [6]. Furthermore, camera-based detection may be affected by poor lighting, overlapping individuals, and obstacles that block the camera's view [22]. These limitations can affect the reliability of crowd counting and occupancy estimation.

Another important problem is that the total number of people in an area does not always represent the actual level of congestion at a particular location. For example, a large open area may accommodate many people, whereas a narrow entrance may become congested with a comparatively smaller number of individuals. Therefore, crowd monitoring requires the identification of specific monitoring zones and the evaluation of occupancy conditions according to the characteristics of each zone.

A camera-based monitoring system can estimate the number of people visible within a monitored area, but an independent sensor can provide additional information about movement through a particular point [20], [21]. An IR/laser break-beam sensor can detect when an object interrupts its beam; however, it cannot independently determine how many people are present throughout a larger area [20], [21]. Integrating camera-based observations and sensor events into a common decision-making process is therefore an important challenge.

Furthermore, monitoring only the current crowd count may not be sufficient to identify developing congestion. A crowd that is currently within its configured safety limit may become overcrowded if people continue entering the area rapidly. Without analysing changes in occupancy over time, the system may fail to identify this developing risk early enough to support preventive action.

To address these problems, the proposed CrowdGuard AI: An Intelligent Crowd Density Monitoring and Alert System for Safe Public Gatherings aims to develop an integrated monitoring framework using Artificial Intelligence (AI), computer vision, and Internet of Things (IoT) technologies.

The system uses the YOLOv8n object detection model to detect people in video frames and ByteTrack to maintain their identities across consecutive frames. It identifies a predefined monitoring zone and calculates the number of detected people within that region. An IR/laser break-beam sensor connected to an ESP32 provides additional crossing-event information to support the monitoring process.

The system evaluates crowd occupancy against a configured threshold and analyses changes in crowd conditions over time. When the specified critical condition is reached, a buzzer and LED connected to the ESP32 provide a local alert. This approach aims to reduce dependence on continuous manual observation and provide useful information for identifying potentially unsafe crowd conditions.

The main problem addressed by this project is the lack of an integrated, low-cost monitoring approach that combines real-time person detection and tracking, zone-based crowd counting, independent sensor information, threshold evaluation, and automatic alerts to support crowd safety in public gatherings.

The proposed system is intended to provide early-warning support for crowd management rather than guarantee the prediction or prevention of every stampede. Its effectiveness in real-world environments must be evaluated through further testing under different crowd sizes, lighting conditions, and environmental circumstances.

1.4 MOTIVATION

The primary motivation behind the development of CrowdGuard AI is to improve crowd safety by introducing an intelligent monitoring system that can identify increasing crowd density and generate timely warnings. During temple festivals, religious processions, and other public gatherings, a large number of people may gather in restricted areas.

Continuous monitoring is essential to identify overcrowding and help event organisers take appropriate preventive measures.

Traditional crowd monitoring methods mainly depend on manual observation and CCTV surveillance [6], [21]. Although these methods are useful, they may not always identify rapidly changing crowd conditions in time. Security personnel may have to monitor multiple camera feeds simultaneously, making it difficult to recognise the early stages of congestion [6]. This motivates the development of an automated system that can continuously analyse crowd conditions and provide useful information for decision-making.

Another important motivation is the application of Artificial Intelligence in public safety. Computer vision techniques can analyse video frames and automatically identify people in a monitored area [5]. In CrowdGuard AI, the YOLOv8n model is used for person detection, while ByteTrack is used to track detected individuals across consecutive frames [1], [2], [5]. These techniques help the system estimate the number of people present within a predefined monitoring zone without requiring manual counting.

The integration of IoT-based sensors provides an additional motivation for this project [19], [20], [21]. Camera-based monitoring may be affected by poor lighting, overlapping individuals, or objects obstructing the camera's view [20]. To complement the visual information, the proposed system uses an IR/laser break-beam sensor connected to an ESP32 microcontroller [19], [21]. The sensor records beam-interruption events when people or objects cross its monitored point. Comparing these events with camera-based observations can provide additional information for evaluating the monitored situation.

The need for early warning is another major motivation for CrowdGuard AI**.** A crowd may initially remain within an acceptable occupancy limit but become congested when people continue entering the area rapidly. Therefore, monitoring only the current number of people may not be sufficient. By analysing changes in crowd occupancy over time and comparing them with configured safety thresholds, the system aims to identify developing congestion and support timely intervention.

The project is also motivated by the need for a cost-effective and practical monitoring solution. Expensive crowd surveillance infrastructure may not be accessible to every temple trust, event organiser, or local authority. CrowdGuard AI uses accessible hardware components, including an ESP32 controller, an IR/laser sensor, a buzzer, and a camera, together with computer vision software running on a computer. This modular approach provides a foundation for developing and testing an affordable crowd monitoring prototype.

Furthermore, the project aims to reduce the workload associated with continuous manual surveillance. By displaying information such as the detected people count, crowd density, sensor events, threshold status, and system condition, the monitoring application can help operators understand the situation more easily. A local buzzer and LED provide additional warnings when the configured critical condition is reached.

The proposed system is motivated by the possibility of extending intelligent crowd monitoring to different public environments. Although the initial application focuses on temple festivals and similar gatherings, the underlying approach may also be useful in railway stations, shopping malls, stadiums, public entrances, and other locations where monitoring specific areas is important.

The major motivations for developing CrowdGuard AI are summarised below.

1. **Improving Public Safety:** To support the identification of potentially dangerous crowd conditions in public gathering areas.
2. **Automating People Detection:** To use AI-based computer vision for detecting and counting people.
3. **Monitoring Specific Zones:** To evaluate crowd occupancy in predefined areas instead of relying only on the overall crowd count.
4. **Integrating IoT Sensors:** To complement camera-based observations with independent sensor events.
5. **Supporting Early Warnings:** To analyse changes in occupancy and compare them with configured safety thresholds.
6. **Generating Automatic Alerts:** To activate a buzzer and LED when the specified critical condition is reached.
7. **Reducing Manual Workload:** To provide useful crowd information through an automated monitoring application.
8. **Developing a Cost-Effective Prototype:** To combine accessible hardware and software technologies in a modular system.
9. **Supporting Future Expansion:** To provide a foundation for monitoring additional zones and integrating more sensors in future versions.

In conclusion, CrowdGuard AI is motivated by the need for a practical, intelligent, and cost-effective approach to crowd monitoring. By combining computer vision, person tracking, IoT-based sensing, occupancy analysis, and automatic alerts, the project aims to support the early identification of potentially unsafe crowd conditions. The system is intended to assist security personnel and event organisers in making informed decisions and improving crowd management during public gatherings.

1.5 RESEARCH OBJECTIVES

The main objective of the CrowdGuard AI project is to develop an intelligent crowd density monitoring and alert system that combines Artificial Intelligence (AI), computer vision, and Internet of Things (IoT) technologies to support safe public gatherings. The system aims to detect people, monitor occupancy within a predefined area, analyse changes in crowd conditions, and generate alerts when specified safety conditions are reached.

The proposed system focuses on improving the monitoring of crowded locations such as temple entrances, religious procession routes, queue corridors, and narrow passages. By integrating camera-based person detection with independent sensor information, the system aims to provide a more comprehensive approach to monitoring crowd conditions.

The specific objectives of the project are described below.

1. Real-Time Crowd Monitoring

To develop a system that continuously monitors a designated area using a camera and processes video frames to identify people and evaluate crowd conditions.

2. AI-Based Person Detection

To implement the YOLOv8n object detection model to identify people in video frames and obtain the bounding boxes required for further analysis.

3. Person Tracking

To use the ByteTrack multi-object tracking algorithm to associate detected individuals across consecutive frames and maintain their tracking identities during monitoring.

4. Zone-Based People Counting

To define a specific monitoring region and count the tracked individuals present within that region. This helps the system focus on important locations such as entrances and narrow passages.

5. Crowd Density Estimation

To calculate crowd density using the number of detected people and the area of the monitored zone. This provides a quantitative measure of crowd concentration within the selected region.

The crowd density is calculated using the following formula:

D=NAD=\frac{N}{A}

Where:

- DD = Crowd density in persons per square metre.
- NN = Number of people detected within the monitored zone.
- AA = Area of the monitored zone in square metres.

6. Integration of IoT-Based Sensors

To integrate an IR/laser break-beam sensor with an ESP32 microcontroller to detect beam interruptions and record crossing events. This sensor provides additional information to complement the camera-based monitoring process.

7. Sensor Data Integration

To compare camera-based observations with sensor events using a rule-based approach. The purpose is to identify discrepancies between the two sources and support the reliability assessment of the monitoring process.

8. Occupancy Threshold Monitoring

To configure an occupancy threshold for the monitored zone and evaluate whether the detected crowd level remains within the specified limit. The threshold can be adjusted according to the requirements of the particular monitoring location.

9. Crowd Growth Rate Analysis

To analyse changes in the number of people over a specified time interval. This helps identify situations where crowd occupancy is increasing rapidly, even if the current count has not yet reached the configured critical threshold.

The crowd growth rate is calculated using the following formula:

G=Ct−Ct−ΔtΔtG=\frac{C_t-C_{t-\Delta t}}{\Delta t}

Where:

- GG = Crowd growth rate.
- CtC_t = Number of people at the current time.
- Ct−ΔtC_{t-\Delta t} = Number of people at the previous measurement.
- Δt\Delta t = Time interval between the measurements.

10. Automatic Alert Generation

To activate a buzzer and LED connected to the ESP32 when the configured critical condition is reached. The alert mechanism is intended to inform nearby personnel that the monitored area requires attention.

11. Real-Time Monitoring Dashboard

To display important monitoring information, including the detected people count, crowd density, sensor events, configured threshold, processing speed, and current system status. This allows operators to observe changes in the monitored area.

12. System Performance Evaluation

To evaluate the prototype using appropriate measures, including person detection performance, counting error, processing speed, sensor operation, and alert response time. The evaluation aims to assess the operation of the integrated system under the tested conditions.

Summary of Research Objectives

The overall objective of CrowdGuard AI is to integrate person detection, tracking, zone-based crowd analysis, independent sensor monitoring, threshold evaluation, and automatic alerts into a single monitoring framework. The system aims to support the early identification of potentially unsafe crowd conditions and provide useful information for crowd management during temple festivals and other public gatherings.

1.7 Organization of the Report

This report is organized into different chapters to explain the design, development, implementation, and evaluation of the proposed CrowdGuard AI: An Intelligent Crowd Density Monitoring and Alert System for Safe Public Gatherings. Each chapter focuses on a specific aspect of the project, providing a systematic understanding of the proposed system.

Chapter 1: Introduction

This chapter introduces the CrowdGuard AI project and explains the importance of intelligent crowd monitoring in temples, religious festivals, processions, and other public gatherings. It discusses the challenges in existing crowd management methods, the problem statement, project motivation, research objectives, and major contributions of the proposed work.

Chapter 2: Literature Review

This chapter reviews existing research and technologies related to crowd monitoring, computer vision, object detection, multi-object tracking, IoT-based sensing, and crowd density estimation. It examines the approaches used in previous systems and identifies the limitations that motivate the development of the proposed system.

Chapter 3: System Analysis and Design

This chapter explains the proposed system architecture and its major modules. It describes the sensing, perception, decision-making, and alert-generation components. The chapter also presents the workflow of the system and explains how camera input, the YOLOv8n detection model, ByteTrack tracking, IR/laser sensors, and the ESP32-based alert mechanism work together.

Chapter 4: System Implementation

This chapter describes the implementation of CrowdGuard AI using Python, OpenCV, YOLOv8n, ByteTrack, and the required IoT hardware. It explains video acquisition, person detection, tracking, monitoring-zone selection, occupancy estimation, threshold checking, sensor integration, and local alert generation.

Chapter 5: Results and Discussion

This chapter presents the results obtained from the implemented prototype. It discusses crowd monitoring outputs, detection performance, processing speed, occupancy calculations, and alert response. The chapter also examines the system's performance under the tested conditions and discusses limitations that may affect its operation in real-world environments.

Chapter 6: Conclusion and Future Enhancements

This chapter summarizes the project objectives, implementation, and major outcomes. It explains how the proposed system combines artificial intelligence and IoT-based sensing to support crowd monitoring. It also identifies possible future enhancements, including broader field testing, improved sensor integration, remote notifications, and deployment in larger public gathering areas.

References

This section lists the research papers, technical documentation, and other sources consulted during the development of the project. These references support the technologies, methods, and concepts discussed throughout the report.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Crowd monitoring is an important research area that focuses on observing, counting, and analyzing people in public places [6], [12]. It plays a significant role in improving safety during temple festivals, religious processions, concerts, public celebrations, and other large gatherings [6].

Traditional crowd management methods mainly depend on security personnel, manual observation, and CCTV surveillance [6], [22]. Although these methods help monitor public areas, they can become difficult to manage when several locations must be observed simultaneously [6]. Therefore, researchers have explored artificial intelligence, computer vision, object detection, multi-object tracking, and Internet of Things (IoT) technologies to automate crowd monitoring [19], [22].

Computer vision techniques allow a system to analyze video frames and identify people automatically [5], [22]. Object detection models can locate individuals in an image, while tracking algorithms associate detections across consecutive frames [5], [22]. These techniques help estimate the number of people present in a particular region.

IoT technologies provide additional capabilities by connecting sensors, microcontrollers, and alert devices [19], [21]. For example, an IR or laser break-beam sensor can detect

crossing events at a specific point, while a microcontroller can process the sensor signal and activate a warning device [19], [21].

The CrowdGuard AI project combines these technologies to develop an intelligent crowd monitoring and alert system. It uses YOLOv8n for person detection, ByteTrack for multi-object tracking, an IR/laser sensor for crossing-event detection, and threshold-based logic for monitoring crowd conditions.

This chapter reviews the existing methods and technologies related to the proposed system, discusses their advantages and limitations, and identifies the research gaps addressed by CrowdGuard AI.

2.1.1 Purpose of the Literature Review

The main purposes of the literature review are:

1. To study traditional crowd monitoring methods and their limitations.
2. To understand AI-based person detection techniques.
3. To examine multi-object tracking algorithms used in video surveillance.
4. To study IoT-based crowd monitoring and sensor integration.
5. To understand crowd density estimation and region-based monitoring.
6. To examine sensor fusion methods for improving monitoring decisions.
7. To identify the limitations of existing systems and establish the need for the proposed CrowdGuard AI framework.

2.1.2 Importance of Existing Research

Previous research provides a foundation for developing reliable crowd monitoring systems. Studies on object detection help identify suitable models for detecting people in video streams. Research on tracking algorithms provides methods for maintaining consistent identities across frames [2], [13], [14], [16].

Similarly, research on crowd density estimation helps explain how the number of people can be related to the available area [3], [7]. IoT-based research demonstrates how sensors and electronic devices can be integrated with software systems to generate alerts [19].

By studying these approaches, the proposed project brings together relevant techniques in a single monitoring framework. Rather than claiming that one algorithm solves every crowd management problem, CrowdGuard AI focuses on integrating person detection, tracking, zone occupancy, independent sensor information, threshold evaluation, and local alert-generation.

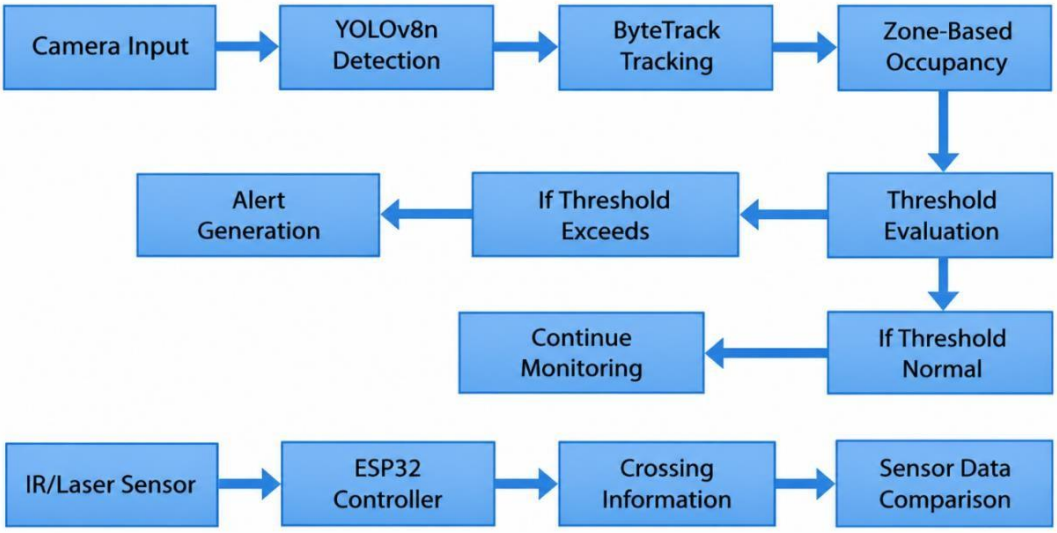


Figure 2.1: Camera and Sensor Processing Flow of CrowdGuard AI

Table 2.1: Comparison of Crowd Monitoring Methods

S. No.	Monitoring Method	Working Principle	Main Advantage
1	Manual Monitoring	Personnel observe crowd movement directly.	Human judgement and contextual awareness.
2	CCTV Surveillance	Cameras provide live video to operators.	Continuous visual monitoring and recorded footage.
3	RFID-Based Monitoring	Readers detect tags carried by people.	Automated identification at checkpoints.
4	IR/Laser Sensor Monitoring	Beam interruption indicates a crossing event.	Simple and economical point-based sensing.
5	AI-Based Video Monitoring	Computer vision detects and tracks people in video.	Automated detection and occupancy estimation.

2.3 AI-Based Crowd Monitoring Systems

Artificial Intelligence (AI) plays an important role in modern crowd monitoring systems by automating the process of detecting people, tracking their movement, and estimating crowd occupancy [1], [6], [22]. Unlike traditional monitoring methods that depend mainly on human observation, AI-based systems analyze video data and convert it into useful information for decision-making [22].

Crowd monitoring is particularly important in temple festivals, religious processions, public celebrations, and other crowded environments. In these locations, the number of people can change rapidly, and certain areas may become congested within a short period. AI-based monitoring helps identify these changes by continuously processing camera input.

Computer vision is one of the major technologies used in AI-based crowd monitoring [23], [27]. It allows computers to extract meaningful information from images and videos [5], [22], [27]. Object detection and multi-object tracking are two important techniques used to identify people and observe their movement over time [5], [22], [23].

The CrowdGuard AI project uses the YOLOv8n object detection model and the ByteTrack tracking algorithm to support automated crowd monitoring. The information obtained from the video is used for zone-based occupancy estimation and threshold checking. An IR/laser sensor provides additional crossing-event information to support the monitoring process.

2.3.1 Computer Vision for Crowd Monitoring

Computer vision is a branch of artificial intelligence that enables computers to analyze and interpret visual information from images and videos [12], [27].

In crowd monitoring, computer vision techniques can identify people, locate them within a camera frame, and provide information about their positions [5], [12]. This information can then be used to estimate occupancy within a selected monitoring area [4].

The general computer vision process includes the following steps:

1. **Image acquisition:** A camera captures video of the monitored environment.
2. **Frame processing:** The video is converted into individual frames for analysis.
3. **Person detection:** An object detection model identifies people in each frame.
4. **Person tracking:** A tracking algorithm associates detections across consecutive frames.
5. **Zone identification:** The system identifies the region selected for monitoring.
6. **People counting:** The system estimates the number of people within the selected region.
7. **Threshold checking:** The estimated occupancy is compared with the configured limit.
8. **Alert generation:** An alert is generated when the required conditions are satisfied.

Computer vision helps automate the extraction of crowd information from video. However, its performance depends on factors such as lighting, camera placement, image quality, and the visibility of individuals.

2.3.2 Object Detection Using YOLO

You Only Look Once (YOLO) is a family of object detection models designed to identify and locate objects in images and video frames [13], [14].

Object detection involves two main tasks:

- **Object classification:** Identifying the type of object present in an image.
- **Object localization:** Determining the position of the object using a bounding box.

In crowd monitoring, the detection model identifies people and generates bounding boxes around the detected individuals [5], [23]. These detections provide the information required for subsequent tracking and counting [5], [23].

YOLO is suitable for real-time applications because its detection process is designed to process images efficiently [1], [13], [14], [17]. Different versions and model sizes provide different trade-offs between processing speed, accuracy, and computational requirements.

The CrowdGuard AI project uses YOLOv8n, the nano variant of YOLOv8. It was selected as a lightweight model for the project's person detection pipeline [1], [14], [18]. The model produces bounding boxes and confidence scores for detected people [18].

The detected people are then passed to the tracking stage for further analysis.

Advantages of YOLO-Based Detection

1. Detects multiple people in a single video frame.
2. Provides bounding boxes indicating the positions of detected people.
3. Supports continuous video analysis.
4. Can be integrated with object tracking algorithms.
5. Offers a lightweight model option suitable for prototype development.

Limitations of YOLO-Based Detection

1. People may be missed when they are heavily occluded.
2. Poor lighting can reduce detection performance.
3. Detection quality depends on image resolution and camera placement.
4. Closely packed people may be difficult to distinguish.
5. Processing performance depends on the available computing hardware.

In CrowdGuard AI, YOLOv8n provides the person detections required for estimating occupancy within the selected monitoring zone.

2.3.3 Multi-Object Tracking

Object detection identifies people in individual video frames, but it does not automatically maintain their identities across time. Multi-object tracking addresses this limitation by associating detections from consecutive frames [15], [16].

Tracking helps the system follow the movement of detected people and maintain temporary track identities while they remain visible [2], [16].

For example, when a person walks across a monitored corridor, the detection model may identify that person in many consecutive frames. A tracking algorithm associates these detections so that the person can be followed over time rather than treated as an entirely new detection in every frame.

The CrowdGuard AI project uses ByteTrack for multi-object tracking.

ByteTrack associates detected bounding boxes across video frames and uses detection confidence informations during the association process [2]. This helps maintain useful tracks when some detections have lower confidence.

Importance of Tracking in Crowd Monitoring

Tracking is useful for the following reasons:

1. Maintains track identities across consecutive frames.
2. Helps analyze the movement of people.
3. Supports more consistent occupancy estimation.
4. Reduces the risk of counting every frame-level detection as a separate person.
5. Provides tracking information that can be used in region-based monitoring.

Tracking does not eliminate every counting error [15]. People may disappear behind other individuals, leave the camera's field of view, or become difficult to distinguish in dense crowds [15], [29]. Therefore, tracking performance must be evaluated under the conditions in which the system will operate.

2.3.4 Region-Based People Counting

Region-based people counting estimates the number of people present within a selected part of the camera view [4], [7], [11], [24].

Instead of counting every detected person in the entire frame, the system defines a region of interest (ROI) corresponding to the area that needs to be monitored [4], [25], [26].

For example, a temple entrance may be selected as a monitoring region because crowd congestion at that location could affect the movement of visitors.

The system identifies tracked people whose selected reference points lie within the defined region. The number of qualifying tracks is then used to estimate occupancy.

The estimated crowd density can be calculated using:

D=NA\boxed{D=\frac{N}{A}}

Where:

- DD = Crowd density in persons per square metre.
- NN = Estimated number of people in the monitored region.
- AA = Area of the monitored region in square metres.

Region-based monitoring allows different areas to have their own occupancy thresholds [4], [7]. For example, a narrow passage may require a different threshold from a large waiting area [7].

In CrowdGuard AI, region-based counting is combined with threshold checking to identify when the estimated occupancy reaches a configured limit.

2.3.5 Real-Time Crowd Monitoring

Real-time crowd monitoring involves continuously processing incoming information and updating the system's understanding of the monitored environment [1], [5].

In an AI-based system, the camera provides video frames that are processed by the detection and tracking algorithms. The resulting information is used to update the estimated crowd count and occupancy status [1], [2].

CrowdGuard AI uses a continuous monitoring workflow:

1. Acquire video frames from the camera.
2. Process the frames using OpenCV.
3. Detect people using YOLOv8n.
4. Track people using ByteTrack.
5. Estimate occupancy within the selected monitoring zone.
6. Compare occupancy with the configured threshold.
7. Update the monitoring status.
8. Generate a local alert when the configured critical condition is reached.

The system also considers changes in occupancy over time. A rapidly increasing crowd count can serve as an additional warning indicator, although it does not independently predict that a dangerous incident will occur.

The practical responsiveness of real-time monitoring depends on processing speed, sensor communication, and the time required to evaluate and respond to the detected conditions [1], [5].

2.3.6 Role of AI in CrowdGuard AI

The proposed CrowdGuard AI system uses AI primarily for person detection and tracking. It combines these functions with zone-based occupancy estimation and IoT-based sensing [1], [19].
The following table summarizes the role of each component.

Table 2.2: Role of Each Component in CrowdGuard AI

Component	Technology Used	Function
Video acquisition	Camera and OpenCV	Captures and processes video frames.
Person detection	YOLOv8n	Detects people and generates bounding boxes.
Person tracking	ByteTrack	Associates detections across consecutive frames.
Region monitoring	ROI-based logic	Identifies people within the selected monitoring zone.
Occupancy estimation	Counting logic	Estimates the number of people in the selected region.
Sensor monitoring	IR/laser break-beam sensor	Detects crossing events at a specific point.
Sensor control	ESP32	Processes sensor signals and supports local alert operation.
Decision-making	Threshold and trend logic	Evaluates occupancy and changes in crowd count.

Alert generation	Buzzer and LED	Provides a local warning when the configured conditions are met.

The camera-based processing and sensor-based monitoring paths provide complementary information. The system compares their outputs to support decision-making rather than simply adding the camera count and sensor event count.

2.3.7 Limitations of AI-Based Crowd Monitoring

Although AI-based crowd monitoring offers several advantages, some challenges remain.

1. Occlusion: People may hide one another when the crowd becomes dense, reducing detection and tracking reliability [29].

2. Lighting conditions: Poor lighting, shadows, and changing illumination can affect the quality of video analysis [23].

3. Camera perspective: The position and angle of a camera influence how clearly people can be detected and how accurately the monitored region can be defined.

4. Computational requirements: Continuous video processing requires sufficient computing resources to maintain an acceptable processing speed [5], [7].

5. Counting errors: Missed detections, false detections, and tracking errors can affect the estimated crowd count [10], [12], [30].

6. Limited sensor coverage: An IR/laser sensor provides crossing information at a particular point and cannot independently describe the crowd distribution throughout a larger area.

7. Need for real-world validation: Performance demonstrated in a controlled environment may not directly represent performance during a large temple festival or public procession.

These limitations indicate the need for careful system configuration and further testing before deployment in large public gatherings.

Summary

AI-based crowd monitoring uses computer vision techniques to automate the detection, tracking, and counting of people. YOLOv8n provides person detections, while ByteTrack associates those detections across video frames. Region-based counting then supports occupancy estimation within a selected monitoring area.

CrowdGuard AI combines these techniques with IR/laser sensor information, threshold-based decision-making, and local alerts. This integration provides a foundation for intelligent crowd monitoring while recognizing that performance in large and complex public gatherings requires further validation.

2.4 IoT-Based Crowd Monitoring Systems

The Internet of Things (IoT) is a technology that connects physical devices, sensors, microcontrollers, and software applications to collect and exchange information [19], [21]. IoT-based systems are widely used in monitoring applications because they allow physical events to be detected and converted into electronic signals for processing [19], [21].

In crowd management, IoT technology can be used to monitor movement, detect people crossing specific points, communicate sensor information, and activate warning devices [19], [20], [21]. When combined with artificial intelligence, IoT provides additional information that can support automated crowd monitoring [19], [21].

Traditional camera-based systems primarily depend on visual information [20]. However, camera performance can be affected by poor lighting, occlusion, and camera positioning [20]. IoT sensors can provide complementary information about specific events, such as movement through an entrance.

The CrowdGuard AI project combines camera-based person detection and tracking with an IR/laser break-beam sensor connected to an ESP32 microcontroller. The camera-processing system estimates occupancy within a selected region, while the sensor detects crossing events. The system uses this information to support threshold evaluation and local alert generation.

2.4.1 Introduction to IoT in Crowd Monitoring

IoT-based crowd monitoring involves connecting sensors and electronic devices to a monitoring application [21]. These devices collect information about the environment and make it available for processing.

For example, an IR/laser sensor installed near an entrance can detect when its light beam is interrupted. A microcontroller receives the sensor signal and processes the event. The information can then be used by the monitoring system to compare sensor events with camera-based observations.

The major components of an IoT-based crowd monitoring system include:

1. **Sensors:** Detect physical events such as beam interruptions.
2. **Microcontroller:** Reads sensor signals and controls connected devices.
3. **Communication interface:** Transfers information between the sensor controller and the monitoring application when configured.
4. **Processing system:** Analyzes the collected information and evaluates monitoring conditions.
5. **Alert devices:** Provide a warning when the required conditions are satisfied.

These components work together to connect physical crowd movement with software-based monitoring and decision-making.

2.4.2 IR/Laser Break-Beam Sensor

An IR/laser break-beam sensor is an electronic sensing arrangement that detects an interruption in a beam of light.

The arrangement generally consists of a light source and a receiver. They are positioned so that the receiver can detect the beam produced by the source.

When a person crosses the path between them, the beam is interrupted. The receiver's output changes, allowing the connected controller to identify a crossing event.

Working Principle

The working process consists of the following steps:

1. The light source produces a beam directed towards the receiver.
2. The receiver continuously monitors the beam.
3. A person or object passes through the beam's path.
4. The beam is interrupted.
5. The sensor output changes.
6. The microcontroller reads the signal and registers the event according to the implemented logic.

The sensor provides information about movement through a specific point. It does not independently identify the person or determine the number of people throughout a larger area.

Advantages

- Simple operating principle.
- Can detect crossing events without identifying individuals.
- Can be connected to a microcontroller.
- Suitable for monitoring specific passages.
- Provides information independent of camera-based person detection.

Limitations

- Detects events only at the installed sensing point.
- Multiple people crossing together may cause counting errors.
- Incorrect alignment can affect operation.
- The sensor cannot independently estimate crowd density across an entire zone.
- A beam interruption does not necessarily represent one unique person.

In CrowdGuard AI, the IR/laser sensor provides complementary crossing-event information for the monitoring process.

2.4.3 ESP32 Microcontroller

The ESP32 is a microcontroller platform that supports electronic input and output operations and provides communication capabilities [21]. It is suitable for IoT prototypes because it can read sensor signals and control connected devices [19], [21].

In CrowdGuard AI, the ESP32 is used to process the signal from the IR/laser sensor and support the local alert mechanism.

Functions of the ESP32

1.SensorInput:
The ESP32 reads the output signal from the connected IR/laser receiver.

2.EventProcessing:

The controller interprets changes in the sensor signal as crossing events according to the programmed logic.

3.DeviceControl:

The ESP32 can control connected output devices, such as a buzzer and LED, when the configured alert conditions are met.

4.CommunicationSupport:

The ESP32 supports communication with other devices and applications through suitable interfaces. The exact communication method depends on the implemented system configuration.

Advantages

- Supports digital input and output operations.
- Can be integrated with common electronic sensors.
- Supports local device control.
- Provides communication capabilities for IoT applications.
- Is suitable for developing compact monitoring prototypes.

The ESP32 provides the interface between the physical sensor and the electronic monitoring system.

2.4.4 Buzzer and LED Alert Mechanism

An alert mechanism communicates important system conditions to nearby personnel. CrowdGuard AI uses a buzzer and LED to provide local warnings.

The buzzer produces an audible signal, while the LED provides a visual indication. These devices can help communicate an alert when the system detects a configured critical condition.

Working Process

1. The system processes the camera and sensor information.
2. The decision logic evaluates the monitored occupancy against the configured threshold.
3. When the alert condition is satisfied, the system initiates the alert response.
4. The buzzer produces an audible warning.
5. The LED provides a visual indication.
6. Personnel can investigate the monitored area and take appropriate action.

The alert devices do not independently determine whether a crowd is dangerous. Their activation depends on the system's programmed decision logic.

Important: The current prototype focuses on local alerts. Remote mobile notifications and cloud-based alert delivery can be considered future enhancements.

2.4.5 Integration of IoT Sensors with AI

Integrating IoT sensors with artificial intelligence allows a monitoring system to use information from more than one sensing method [20].

In a camera-based system, YOLOv8n detects people in video frames, and ByteTrack associates detections across frames [1], [2]. The resulting tracking information supports occupancy estimation within a selected monitoring zone.

The IR/laser sensor provides crossing-event information through the ESP32. These two information sources have different purposes and measurement characteristics [20].

The camera estimates people present within a region, whereas the beam sensor records interruptions at a specific point [20]. Therefore, their outputs should not simply be added together.

CrowdGuard AI uses a decision-oriented comparison to examine the relationship between the camera count and sensor events.

Let:

- CC = Camera-based count.
- LL = Number of sensor crossing events during the observation window.

The discrepancy can be represented as:

E=∣C−L∣ \boxed{E=|C-L|}

Where EE represents the absolute difference between the two values.

A large discrepancy may indicate that the measurements require further examination. However, this difference is not a universal measure of sensor accuracy because the camera and beam sensor may observe different areas or time intervals [20].

The comparison therefore supports consistency checking rather than providing a guaranteed correction to the crowd count.

2.4.6 Role of IoT in CrowdGuard AI

The following table summarizes the main hardware components and their functions.

Table 2.3: Hardware Components and Their Functions

S. No.	Component	Function in the Proposed System
1	Camera	Provides video input for crowd monitoring.
2	IR/Laser Sensor	Detects interruptions in the light beam.
3	ESP32	Processes sensor signals and supports local device control.
4	Buzzer	Provides an audible local warning.
5	LED	Provides a visual local warning.
6	Computer or Laptop	Runs the AI-based video processing and monitoring application.

The camera-based AI processing is performed on the computer in the current prototype, while the ESP32 handles the sensor-side operations and supports the local response mechanism.

This separation allows the system to combine computer vision with electronic sensing without requiring every processing task to run directly on the microcontroller.

2.4.7 Advantages of IoT-Based Crowd Monitoring

IoT-based monitoring provides several advantages when used alongside AI.

1. Automated event detection: Sensors can detect physical events without continuous manual observation.

2. Complementary information: Sensor signals can provide information that is different from camera-based estimates.

3. Local response: Connected output devices can provide warnings when configured conditions are met.

4. Modular design: Additional sensors may be incorporated into the architecture when required.

5. Practical implementation: Common electronic components can be used to develop and test a prototype.

6. Support for decision-making: Sensor information can be compared with camera observations to identify discrepancies.

However, these advantages depend on correct hardware configuration, appropriate event-processing logic, and reliable communication between the components.

2.4.8 Limitations of IoT-Based Crowd Monitoring

Despite its advantages, IoT-based crowd monitoring has certain limitations.

- Sensors provide information only about the events they are designed to detect.
- Hardware alignment and installation affect sensor performance.
- Communication failures may interrupt information transfer.
- Incorrect event processing may produce inaccurate counts.
- A single sensor cannot describe crowd distribution across a large area.
- Sensor readings must be interpreted in relation to the camera's monitoring region and observation period.

For these reasons, CrowdGuard AI combines sensor information with camera-based occupancy estimation instead of relying exclusively on either source.

Summary

IoT technology enables physical sensors and electronic devices to contribute information to automated crowd monitoring. The IR/laser break-beam sensor detects crossing events, while the ESP32 processes the sensor signal and supports local alert operation.

CrowdGuard AI combines these capabilities with AI-based person detection, tracking, and zone occupancy estimation. The integration provides complementary information for threshold evaluation and local warnings, although broader testing is necessary to establish reliability in real-world crowd conditions.

2.6 Research Gaps in Existing Crowd Monitoring Systems

Existing crowd monitoring systems use different technologies, including manual observation, CCTV surveillance, artificial intelligence, computer vision, and IoT-based sensors [6], [12], [20]. These technologies help monitor people and identify crowded areas. However, several limitations remain when applying them to temple festivals, religious processions, and other large public gatherings [6].

The literature reviewed in the previous sections highlights the need for a monitoring system that combines person detection, tracking, zone-based occupancy estimation, independent sensor information, and automatic alert generation.

The proposed CrowdGuard AI system addresses these requirements by integrating YOLOv8n, ByteTrack, an IR/laser break-beam sensor, an ESP32 microcontroller, and threshold-based decision logic. The following research gaps explain the motivation for this integration.

2.6.1 Limited Automation in Traditional Monitoring

Traditional crowd monitoring depends heavily on security personnel and manual observation [6]. Although experienced personnel can identify visible problems, monitoring several locations simultaneously can be difficult.

Conventional CCTV systems provide video feeds, but automated crowd analysis requires additional processing capabilities [22].

Research gap: There is a need for automated monitoring that converts video input into useful information, such as the estimated number of people in a selected area.

Proposed solution: CrowdGuard AI uses YOLOv8n for person detection and ByteTrack for tracking people across consecutive video frames.

2.6.2 Limited Integration of Independent Sensors

Many camera-based monitoring approaches primarily depend on visual information [20]. However, camera performance can be affected by poor lighting, occlusion, and camera placement [20].

An independent sensor can provide additional information about physical events, such as movement through a particular entrance. Nevertheless, a beam sensor alone cannot estimate crowd distribution across a larger region.

Research gap: There is a need to combine camera-based occupancy estimation with independent sensor information.

Proposed solution: CrowdGuard AI integrates camera-based analysis with an IR/laser break-beam sensor connected to an ESP32. The sensor provides crossing-event information that complements the camera-based estimate.

2.6.3 Lack of Zone-Based Occupancy Monitoring

A single overall crowd count may not provide enough information about conditions at specific locations. For example, a narrow entrance may become congested even when the total number of people across a larger venue remains within its overall capacity.

Different regions may have different sizes and occupancy requirements.

Research gap: There is a need to monitor specific regions and evaluate their occupancy separately.

Proposed solution: CrowdGuard AI uses region-of-interest logic to estimate the number of people in a selected monitoring zone. The estimated occupancy can then be compared with a configured threshold for that zone.

2.6.4 Limited Consideration of Crowd Growth Rate

Monitoring only the current number of people may not reveal how quickly a crowd is increasing. Two locations may have similar occupancy counts, but one may be experiencing a much faster increase in the number of people.

Tracking changes in occupancy over time can provide additional information for monitoring decisions.

Research gap: There is a need to consider changes in crowd occupancy alongside the current count.

Proposed solution: CrowdGuard AI calculates the rate of change in crowd count using the following formula:

G=Ct−Ct−ΔtΔt\boxed{G=\frac{C_t-C_{t-\Delta t}}{\Delta t}}

Where:

- GG = Rate of change in crowd count.
- CtC_t = Current crowd count.
- Ct−ΔtC_{t-\Delta t} = Crowd count at an earlier time.
- Δt\Delta t = Time interval between the measurements.

This value can support warning decisions when occupancy is increasing rapidly. However, crowd growth rate alone cannot guarantee the prediction of a dangerous incident.

2.6.5 Limited Integration of Monitoring and Physical Alerts

Some monitoring approaches focus primarily on displaying video or estimating crowd counts. However, identifying a condition is only one part of a monitoring system; the information must also be communicated to personnel who can respond.

Research gap: There is a need to connect monitoring decisions with a physical alert mechanism.

Proposed solution: CrowdGuard AI incorporates a buzzer and LED to provide local warnings when the configured critical conditions are satisfied.

The alert mechanism connects the software-based monitoring process with a physical response. Remote notifications and mobile alerts remain potential future enhancements.

2.6.6 Challenges in Sensor Data Interpretation

Different sensors may produce different types of information [20]. A camera estimates the number of people currently present within a region, while an IR/laser sensor records beam interruptions during an observation period [20].

These measurements cannot always be compared directly because they may represent different locations, events, and time intervals [20].

Research gap: There is a need for a suitable method to compare different sensor outputs without incorrectly treating them as equivalent measurements.

Proposed solution: CrowdGuard AI uses decision-level sensor comparison. It examines the difference between the camera-based count and the sensor-event count to identify possible,discrepancies.

The comparison is used as a consistency-checking indicator rather than a universal measurement of sensor accuracy.

2.6.7 Need for a Modular and Adaptable Architecture

Crowd monitoring requirements vary according to the size and layout of the monitored environment. An entrance, a corridor, and a large waiting area may require different monitoring configurations.

A system that tightly couples all its components can be difficult to modify when new sensors or monitoring zones are introduced.

Research gap: There is a need for a modular architecture that separates sensing, perception, decision-making, and response.

Proposed solution: CrowdGuard AI organizes its functions into separate modules:

1. **Sensing:** Acquires camera input and sensor events.
2. **Perception:** Detects and tracks people using YOLOv8n and ByteTrack.
3. **Decision-making:** Evaluates occupancy, growth rate, and configured thresholds.
4. **Response:** Activates local alert devices when required.

This architecture provides a foundation for future expansion, including additional sensors and monitoring zones.

2.6.8 Limited Validation in Real-World Public Gatherings

A system that performs successfully in a controlled demonstration may not achieve the same performance in a large public gathering [8]. Real-world environments introduce challenges such as changing lighting, dense crowds, occlusion, weather conditions, and communication problems [8].

Temple festivals and religious processions can also involve changing crowd movement patterns and temporary changes to access routes [8].

Research gap: There is a need for broader evaluation under realistic crowd conditions before an automated monitoring system can be considered suitable for large-scale deployment.

Proposed solution: CrowdGuard AI provides a prototype for integrating computer vision, sensor information, occupancy evaluation, and local alerts. Further testing in different environments is necessary to establish its reliability and practical effectiveness.

2.6.9 Summary of Research Gaps

The following table summarizes the research gaps and the corresponding approach used in CrowdGuardAI.

Table 2.4: Research Gaps and Proposed Approach

S. No.	Research Gap	Proposed Approach in CrowdGuard AI
1	Dependence on manual observation	Automated person detection using YOLOv8n
2	Difficulty maintaining consistent person observations across frames	Multi-object tracking using ByteTrack
3	Limited monitoring of specific locations	Region-of-interest-based occupancy estimation
4	Lack of independent sensor information	Integration of an IR/laser break-beam sensor
5	Limited consideration of changing occupancy	Crowd growth-rate calculation
6	Difficulty interpreting different sensor outputs	Decision-level sensor comparison
7	Lack of a direct local warning mechanism	Buzzer and LED alert generation
8	Difficulty adapting systems to different locations	Modular sensing, perception, decision, and response architecture
9	Insufficient real-world validation	Need for broader testing under different crowd and environmental conditions

Conclusion

The literature review identifies the need for a crowd monitoring system that combines AI-based person detection, multi-object tracking, zone-based occupancy estimation, independent sensor information, and automatic local alerts.

CrowdGuard AI addresses these research gaps through a modular framework that integrates YOLOv8n, ByteTrack, an IR/laser sensor, an ESP32, and threshold-based decision logic.

The proposed system provides a preliminary functional approach to automated crowd monitoring. However, its performance in large-scale public gatherings must be established through further testing under realistic conditions.

2.7 Summary of Literature Review

This chapter reviewed existing technologies and research related to crowd monitoring, artificial intelligence, computer vision, IoT-based sensing, and sensor fusion [6], [19], [20]. The review helped identify the limitations of traditional monitoring methods and the need for an integrated system that can estimate crowd occupancy and generate alerts when configured safety thresholds are reached [6], [19].

Traditional crowd monitoring methods, such as manual observation and CCTV surveillance, are useful for observing public areas [6], [21]. However, they may require continuous human attention and may not automatically estimate crowd occupancy or generate warnings. RFID-based monitoring and electronic sensors provide alternative methods for identifying people or detecting movement at specific locations, but each method has its own limitations [20], [21].

Artificial intelligence and computer vision provide opportunities to automate person detection and tracking. In the proposed CrowdGuard AI system, YOLOv8n is used to

detect people in video frames, while ByteTrack associates detections across consecutive frames [1], [2], [14]. These techniques support people counting and occupancy estimation within a selected monitoring zone.

The literature review also examined IoT-based monitoring [19], [21]. The IR/laser break-beam sensor provides crossing-event information, while the ESP32 processes the sensor signals and supports local alert operation [19], [21]. The buzzer and LED provide audible and visual warnings when the configured alert conditions are satisfied.

Sensor fusion was identified as another important aspect of the proposed system [20]. CrowdGuard AI compares camera-based occupancy information with IR/laser sensor events [20]. This comparison supports consistency checking and helps identify discrepancies between the two information sources. The measurements are not simply added together because they may represent different locations, events, or observation periods.

The review identified several research gaps, including limited automation in traditional monitoring, insufficient integration of independent sensors, the need for zone-based occupancy monitoring, limited consideration of crowd growth rate, and the need to connect monitoring decisions with local alerts. It also highlighted the importance of broader testing under realistic crowd conditions.

Key Findings of the Literature Review

Table 2.5: Key Findings of the Literature Review

S. No.	Area Studied	Key Finding	Application in CrowdGuard AI
1	Traditional Crowd Monitoring	Manual observation requires continuous attention.	Automated monitoring support.
2	Computer Vision	Video can be analyzed to detect people automatically.	YOLOv8n person detection.
3	Multi-Object Tracking	Tracking associates detections across consecutive frames.	ByteTrack tracking.
4	Zone-Based Monitoring	Specific regions may require separate occupancy monitoring.	Region-of-interest-based counting.
5	IoT-Based Sensing	Electronic sensors provide information about physical events.	IR/laser sensor and ESP32 integration.
6	Sensor Fusion	Different sensor outputs can provide complementary information.	Camera and sensor-event comparison.
7	Crowd Growth Analysis	Changes in occupancy provide additional monitoring information.	Crowd growth-ratecalculation.
8	Alert Mechanisms	Monitoring results can be connected to physical warning devices.	Buzzer and LED activation.
9	System Evaluation	Controlled demonstrations do not establish performance in every real-world environment.	Need for further field testing.

Conclusion

The literature review establishes the technical foundation for the proposed CrowdGuard AI system. By combining AI-based person detection, multi-object tracking, zone-based occupancy estimation, IoT sensor information, threshold evaluation, and local alerts, the project aims to support automated crowd monitoring [1], [2], [19], [20].

The proposed system is a preliminary functional prototype. Its reliability and effectiveness in large public gatherings require further evaluation under different environmental conditions and crowd densities.

The next chapter explains the system analysis and design of CrowdGuard AI, including the proposed architecture, hardware and software components, and the working process of the system.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

System analysis and design is an important phase in the development of the CrowdGuard AI project. It involves understanding the requirements of the proposed system, identifying the limitations of existing approaches, selecting suitable technologies, and designing a workflow that connects all the system components.

The proposed project, **CrowdGuard AI: An Intelligent Crowd Density Monitoring and Alert System for Safe Public Gatherings**, is designed to monitor crowd occupancy and provide warnings when configured thresholds are reached. The system focuses on temple festivals, religious processions, and other public gatherings where crowd movement and overcrowding require careful observation.

Traditional monitoring methods primarily depend on security personnel and CCTV surveillance [6], [21]. Although these methods provide useful information, continuously monitoring several locations and estimating crowd occupancy can be difficult. CrowdGuard AI addresses these limitations by integrating artificial intelligence, computer vision, and IoT-based sensing [21].

The system uses a camera to capture video from the monitored environment. OpenCV processes the video frames, and the YOLOv8n object detection model identifies people in each frame. The ByteTrack tracking algorithm associates detections across consecutive frames, helping the system maintain consistent tracking information for occupancy estimation.

A selected region of interest (ROI) is used to identify the area that requires monitoring. The system estimates the number of people within this region and evaluates the occupancy against a configured threshold. Crowd density and the rate of change in crowd count can provide additional information for monitoring decisions.

In addition to camera-based analysis, the system incorporates an IR/laser break-beam sensor connected to an ESP32 microcontroller. The sensor detects crossing events and provides information that complements the camera-based occupancy estimate. The system compares these sources to identify possible discrepancies and support decision-making.

When the configured critical conditions are satisfied, the system can activate a local buzzer and LED to provide an audible and visual warning. This connects the software-based monitoring process with a physical alert mechanism.

3.1.1 Purpose of System Design

The main purpose of the system design is to establish a clear structure for integrating the AI software, electronic sensors, microcontroller, and alert devices.

The major objectives of the design are:

1. To capture video from the monitored environment.
2. To detect people using YOLOv8n.
3. To track detected people using ByteTrack.
4. To estimate occupancy within a selected monitoring zone.
5. To obtain crossing-event information from an IR/laser sensor.
6. To process sensor signals using the ESP32.
7. To compare camera-based occupancy information with sensor events.
8. To evaluate crowd occupancy against configured thresholds.
9. To monitor changes in crowd count over time.
10. To generate local alerts when the required conditions are satisfied.

3.1.2 Overview of the Proposed System

CrowdGuard AI is organized into four major layers.

Table 3.1: System Layers of CrowdGuard AI

S. No.	System Layer	Main Function	Components or Technologies
1	Sensing Layer	Collects video and crossing-event information.	Camera and IR/laser sensor
2	Perception Layer	Detects and tracks people in video frames.	OpenCV, YOLOv8n, ByteTrack
3	Decision Layer	Estimates occupancy and evaluates monitoring conditions.	ROI logic, sensor comparison, threshold and growth-rate calculations
4	Response Layer	Provides a local warning when required.	ESP32, buzzer and LED

The sensing layer collects information from the monitored environment. The perception layer processes the video and extracts information about people. The decision layer evaluates occupancy and sensor information, while the response layer provides a warning when the configured conditions are met.

The camera-processing pipeline runs on a computer or laptop in the current prototype. The ESP32 handles sensor-side operations and supports the local alert mechanism.

3.1.3 Working Principle

The proposed system follows a continuous monitoring process.

First, the camera captures video of the monitored area. OpenCV reads and processes the incoming frames, which are passed to YOLOv8n for person detection.

The detected people are then processed by ByteTrack, which associates detections across consecutive frames. The system uses the tracking information to estimate occupancy within the selected monitoring region.

At the same time, the IR/laser sensor detects interruptions in its light beam. The ESP32 processes the sensor signals and registers crossing events.

The system compares the camera-based count with the sensor-event information. It then evaluates the estimated occupancy against the configured threshold and considers changes in crowd count over time.

If the required critical condition is satisfied, the system activates the local alert mechanism. Otherwise, monitoring continues.

3.1.4 Importance of the Proposed Design

The proposed design integrates several technologies into a single crowd monitoring framework. Person detection and tracking provide information about people in the camera view, while the IR/laser sensor supplies complementary crossing-event information.

Zone-based occupancy estimation helps focus monitoring on specific locations instead of relying only on an overall crowd count. Threshold evaluation and crowd growth-rate analysis provide additional information for identifying conditions that may require attention.

The local buzzer and LED allow the system to communicate warnings without depending exclusively on the monitoring application's display.

However, the system is intended to support crowd monitoring rather than guarantee the prediction or prevention of every dangerous incident. Its practical effectiveness depends on detection performance, sensor configuration, and further testing in realistic environments.

Summary

This chapter begins the analysis and design of CrowdGuard AI by explaining its purpose, architecture, major components, and working principle. The system combines computer vision, AI-based detection and tracking, IoT sensing, threshold evaluation, and local alert generation.

3.2 Existing System and Its Limitations

The existing crowd monitoring system mainly relies on manual observation, CCTV surveillance, and separate electronic sensors to monitor people in public places [6], [21]. These methods are used in temple festivals, religious processions, public gatherings, and other crowded environments [6].

Security personnel observe crowd movement, identify congested areas, and take necessary actions when problems occur. CCTV cameras provide visual information, while sensors such as IR/laser break-beam devices can detect movement through specific points.

Although these methods are useful, they have limitations when used independently. Manual observation requires continuous attention, conventional CCTV systems may depend on operators to identify problems, and individual sensors cannot provide complete information about crowd distribution across a larger area [6], [8], [20].

The existing approaches also may not combine occupancy estimation, crowd growth-rate analysis, independent sensor information, and automatic local alerts within a single monitoring workflow [19], [20].

Therefore, CrowdGuard AI proposes an integrated approach using artificial intelligence, computer vision, and IoT-based sensing to support crowd monitoring and warning generation.

3.2.1 Existing System

The existing system can be understood through the following methods.

A. Manual Crowd Monitoring

In manual monitoring, security personnel observe crowd movement directly. They monitor entrances, exits, queues, corridors, and other important locations.

Limitations:

- Requires continuous human attention.
- Monitoring multiple locations simultaneously is difficult.
- Crowd counting may be inconsistent.
- Rapid changes in crowd conditions may not be identified immediately.

B. CCTV-Based Monitoring

CCTV cameras capture video from public areas and display it on monitoring screens [6], [22]. Security personnel observe the footage to identify overcrowding or unusual situations [6].

Limitations:

- Conventional CCTV may not automatically count people.

- Operators must interpret the video.
- Occlusion and poor lighting can affect visibility.
- Separate CCTV feeds may make it difficult to understand occupancy across multiple locations.

C. Sensor-Based Monitoring

Sensor-based systems use electronic sensors to detect specific events [20], [21]. For example, an IR/laser break-beam sensor detects an interruption when a person or object passes through its beam [20], [21].

Limitations:

- Provides information only about the monitored point.
- Cannot independently estimate occupancy across an entire region.
- Multiple people crossing together can complicate event counting.
- Sensor information must be interpreted before it can support crowd-level decisions.

3.2.2 Limitations of the Existing System

The major limitations of existing crowd monitoring approaches are summarized below.

1. Dependence on Manual Observation

Traditional monitoring requires personnel to observe crowd movement and identify potential problems. Continuous monitoring becomes challenging when several entrances and narrow passages must be watched simultaneously.

2. Lack of Automated Person Detection

Conventional surveillance systems may provide video without automatically identifying and counting people. This increases the workload of monitoring personnel.

3. Difficulty in Consistent People Counting

Counting people manually or estimating occupancy from video can be difficult in crowded areas, particularly when people overlap or move quickly.

4. Limited Zone-Based Monitoring

A total crowd count may not reveal congestion in a specific entrance or narrow passage. Individual high-risk regions may require separate occupancy estimates and thresholds.

5. Limited Sensor Integration

A camera and an independent sensor may provide different information, but separate systems do not necessarily compare their outputs. Without suitable integration, discrepancies-unnoticed.

6. Limited Crowd Growth Analysis

Monitoring only the current crowd count does not indicate how quickly occupancy is changing. Changes over time can provide additional information for identifying situations that require attention.

7. Lack of Automatic Local Alerts

Some monitoring systems primarily display information and rely on personnel to identify and communicate the need for action. An integrated alert mechanism can provide a direct local warning when configured conditions are satisfied.

8. Environmental Challenges

Poor lighting, occlusion, camera placement, weather, and communication problems can affect monitoring performance. These challenges are particularly important in outdoor public gatherings.

3.2.3 Comparison Between Existing and Proposed Systems

The following table compares the main characteristics of existing monitoring methods with the proposed CrowdGuard AI approach.

Table 3.2: Comparison of Existing and Proposed Systems

S. No.	Feature	Existing Monitoring Approaches	Proposed CrowdGuard AI
1	Person detection	May depend on manual observation or separate analytics.	Uses YOLOv8n for person detection.
2	Person tracking	May require manual observation or separate tracking software.	Uses ByteTrack to associate detections across frames.
3	People counting	May involve manual estimates or individual counting methods.	Estimates occupancy from tracked detections within a selected region.
4	Zone monitoring	May depend on camera placement and operator observation.	Uses region-of-interest-based monitoring.
5	Sensor information	May be processed independently.	Compares camera-based occupancy with IR/laser crossing events.
6	Crowd growth analysis	May not be included in conventional monitoring.	Calculates the rate of change in crowd count.
7	Threshold checking	May depend on operator judgement or separate systems.	Evaluates occupancy against configured thresholds.
8	Local alerts	May require manual communication or separate devices.	Supports local buzzer and LED alerts.
9	System architecture	May use separate monitoring components.	Integrates sensing, perception, decision-making, and response modules.

Note: The comparison describes general characteristics of the monitoring approaches discussed in the project paper. Not every existing system has all the listed limitations; advanced surveillance systems may already provide some of these capabilities.

3.2.4 Need for the Proposed System

The limitations discussed above establish the need for an integrated crowd monitoring framework.

CrowdGuard AI combines camera-based person detection, tracking, region-based occupancy estimation, sensor-event comparison, threshold checking, and local alert generation.

The system follows these major steps:

1. Captures video from the monitored environment.
2. Detects people using YOLOv8n.
3. Tracks detected people using ByteTrack.
4. Estimates occupancy within the selected monitoring region.
5. Obtains crossing-event information from the IR/laser sensor.
6. Processes sensor signals through the ESP32.
7. Compares the camera-based count with sensor-event information.
8. Evaluates occupancy against the configured threshold.
9. Activates the local alert mechanism when the required conditions are met.

This integrated approach is intended to support monitoring personnel by providing automated occupancy information and local warnings.

However, the proposed system is a prototype rather than a replacement for security personnel or established crowd-management procedures. Its performance in large public gatherings requires further testing.

Summary

The existing system relies on manual observation, CCTV surveillance, or individual sensors. These methods provide useful information but may not independently offer all the capabilities required for automated crowd monitoring.

CrowdGuard AI addresses these limitations by integrating AI-based person detection and tracking with IoT sensing, zone-based occupancy estimation, threshold evaluation, and local alerts.

3.3 Proposed System

The proposed system, CrowdGuard AI: An Intelligent Crowd Density Monitoring and Alert System for Safe Public Gatherings, is designed to monitor crowd occupancy and generate alerts when configured safety thresholds are reached. The system combines artificial intelligence, computer vision, IoT-based sensing, and threshold-based decision-making to support crowd monitoring in temple festivals, religious processions, and other publicgatherings.

The proposed system addresses limitations associated with relying exclusively on manual observation, conventional CCTV surveillance, or individual sensors [19], [20]. It integrates camera-based person detection and tracking with independent sensor information to provide a more comprehensive view of the monitored environment [19], [20].

CrowdGuard AI uses YOLOv8n to detect people in video frames and ByteTrack to associate detections across consecutive frames [1], [2], [14]. The resulting tracking information supports people counting within a selected region of interest (ROI). The system uses this information to estimate occupancy and compare it with a configured threshold.

In parallel, an IR/laser break-beam sensor detects crossing events and sends its signal to an ESP32 microcontroller. The sensor information is compared with the camera-based occupancy estimate to identify possible discrepancies and support the monitoring decision.

When the configured critical conditions are satisfied, the system activates a local buzzer and LED to provide an audible and visual warning. The computer-based monitoring application displays information about the crowd and the system's current status.

3.3.1 Objectives of the Proposed System

The main objectives of the proposed system are:

1. To monitor people in a selected area using a camera.
2. To detect people automatically using YOLOv8n.
3. To track detected people using ByteTrack.
4. To estimate occupancy within a defined monitoring zone.
5. To calculate crowd density based on occupancy and area.
6. To detect crossing events using an IR/laser break-beam sensor.
7. To process sensor signals using an ESP32 microcontroller.
8. To compare camera-based occupancy information with sensor events.
9. To calculate the rate of change in crowd count.
10. To evaluate occupancy against configured thresholds.
11. To display crowd information through the monitoring application.
12. To generate local alerts when the required conditions are satisfied.

3.3.2 Architecture of the Proposed System

The proposed system consists of four major layers: sensing, perception, decision-making, and response.

1. Sensing Layer

The sensing layer collects information from the monitored environment.

It includes:

- Camera or webcam for video acquisition.
- IR/laser break-beam sensor for crossing-event detection.

The camera captures video of the monitored area, while the IR/laser sensor detects interruptions in its beam.

2. Perception Layer

The perception layer processes video input and extracts information about people.

It includes:

- OpenCV for video and frame processing.
- YOLOv8n for person detection.
- ByteTrack for multi-object tracking.

This layer provides the detections and tracking information required for occupancy estimation.

3. Decision-Making Layer

The decision-making layer interprets the information obtained from the camera and sensor.

Its functions include:

- Region-of-interest identification.
- People counting.
- Crowd density estimation.
- Sensor data comparison.
- Crowd growth-rate calculation.
- Threshold evaluation.
- Monitoring status determination.

The system evaluates the estimated occupancy against the configured threshold and uses the resulting status to determine whether an alert is required.

4. Response Layer

The response layer communicates important monitoring conditions.

It includes:

- Buzzer for an audible warning.
- LED for a visual warning.
- Monitoring dashboard for displaying system information.

When the required critical condition is satisfied, the local alert mechanism is activated.

3.3.3 Working Principle of the Proposed System

The proposed system operates through a sequence of interconnected steps.

Step 1: Video Acquisition

The camera captures video of the selected monitoring area. The video provides the visual information required for person detection and tracking.

Step 2: Frame Processing

OpenCV reads the video stream and prepares individual frames for analysis.

Step 3: Person Detection

YOLOv8n processes the frames and identifies people [1], [14]. It generates bounding boxes around detected individuals.

Step 4: Person Tracking

ByteTrack associates detections across consecutive frames [2]. This provides tracking information that helps the system monitor people as they move through the camera view.

Step 5: Monitoring Zone Identification

The system identifies the selected region of interest. Only detections satisfying the implemented zone-selection logic are considered for occupancy estimation.

Step 6: People Counting

The system estimates the number of tracked people within the selected region.

Step 7: Sensor Event Detection

The IR/laser sensor detects interruptions in its beam. The ESP32 processes the sensor signal and registers crossing events.

Step 8: Sensor Data Comparison

The camera-based occupancy estimate and sensor-event information are compared. The system considers their relationship rather than simply adding the two values together.

Step 9: Crowd Condition Evaluation

The system evaluates occupancy against the configured threshold. Changes in crowd count over time can also provide additional monitoring information.

Step 10: Alert Generation

If the configured critical condition is satisfied, the system activates the local buzzer and LED. Otherwise, the monitoring process continues.

3.3.4 Crowd Density Calculation

Crowd density represents the number of people per unit area.

It is calculated using the formula:

D=NA\boxed{D=\frac{N}{A}}

Where:

- DD = Crowd density in persons per square metre.
- NN = Estimated number of people in the monitored area.
- AA = Area of the monitored region in square metres.

Example:

Suppose the monitored region has an area of 100 m2100\,m^2 and the estimated number of people is 20.

Then:

D=20100D=\frac{20}{100} D=0.2 persons/m2\boxed{D=0.2\text{ persons/m}^2}

This value represents the average number of people per square metre within the selected region.

The calculated density depends on the accuracy of the people count and the area measurement. It should not be treated as a complete measure of crowd safety because movement patterns and local congestion can also be important.

3.3.5 Crowd Growth-Rate Calculation

Crowd growth rate measures how quickly the estimated number of people changes over a period of time.

The formula is:

G=Ct−Ct−ΔtΔt\boxed{G=\frac{C_t-C_{t-\Delta t}}{\Delta t}}

Where:

- GG = Rate of change in crowd count.
- CtC_t = Current crowd count.
- Ct−ΔtC_{t-\Delta t} = Crowd count at the earlier time.
- Δt\Delta t = Time interval between the two measurements.

Example:

Suppose the crowd count increases from 30 to 50 people over two minutes.

G=50−302G=\frac{50-30}{2} G=10 people per minute\boxed{G=10\text{ people per minute}}

This indicates that the estimated crowd count increased at an average rate of 10 people per minute during the observed interval.

Crowd growth rate can help identify rapidly increasing occupancy. However, it is an indicator for monitoring and warning, not a guarantee that an unsafe event will occur.

3.3.6 Sensor Data Comparison

The proposed system uses information from two sensing sources:

1. Camera-based occupancy estimation.
2. IR/laser crossing-event detection.

Let:

- CC = Camera-based count.
- LL = Number of sensor crossing events during the observation window.

The absolute difference can be expressed as:

E=∣C−L∣ \boxed{E=|C-L|}

Where EE represents the discrepancy between the two values.

For example, if the camera estimates 25 people and the sensor records 22 crossing events during the selected observation window:

E=∣25−22∣=3E=|25-22|=3

The discrepancy is 3.

However, this does not automatically mean that three people were missed. The camera may estimate the number of people currently present in a region, whereas the sensor records crossings over time. The comparison must therefore be interpreted according to the sensor placement and observation interval.

The proposed system uses this comparison as a consistency-checking indicator to support monitoring-decisions.

3.3.7 Threshold-Based Alert Generation

Threshold-based monitoring allows the system to evaluate whether the estimated occupancy has reached a configured limit.

Each monitoring zone can have an occupancy threshold according to its size and intended use.

The general decision process is:

- **Normal condition:** Occupancy remains within the configured limit, and monitoring continues.
- **Warning condition:** Occupancy approaches a configured warning level or other warning criteria are met.
- **Critical condition:** The configured critical condition is satisfied, and the local alert mechanism is activated.

The system can represent warning states using Green, Yellow, Orange, and Red levels.

The exact threshold values must be configured for the monitored environment. The system does not independently establish legally approved occupancy limits.

3.3.8 Advantages of the Proposed System

The major advantages of CrowdGuard AI are:

1. **Automated Monitoring:** Uses AI-based person detection to reduce dependence on continuous manual counting.
2. **Person Tracking:** Uses ByteTrack to associate detections across consecutive frames.
3. **Zone-Based Occupancy:** Supports monitoring of specific areas within the camera view.
4. **Complementary Sensor Information:** Integrates camera-based analysis with IR/laser crossing events.
5. **Crowd Trend Analysis:** Calculates changes in crowd count over time.
6. **Threshold-Based Decisions:** Compares occupancy with configured limits.
7. **Local Alerts:** Uses a buzzer and LED to communicate critical monitoring conditions.
8. **Modular Design:** Separates sensing, perception, decision-making, and response.
9. **Expandable Architecture:** Provides a foundation for adding monitoring zones or additional sensing capabilities.

3.3.9 Limitations of the Proposed System

Although CrowdGuard AI integrates several useful monitoring capabilities, it has limitations.

- Occlusion may reduce the accuracy of person detection and tracking.

- Poor lighting can affect camera-based analysis.
- Incorrect camera placement can reduce monitoring effectiveness.
- Multiple people crossing the beam simultaneously may complicate sensor-event counting.
- Communication problems may affect the transfer of sensor information.
- Incorrect threshold configuration can produce unsuitable warnings.
- Performance in a controlled demonstration may differ from performance during a large public gathering.

Therefore, further evaluation under different crowd densities and environmental conditions is required before large-scale deployment.

Summary

The proposed CrowdGuard AI system integrates AI-based person detection, multi-object tracking, zone-based occupancy estimation, IR/laser sensor information, threshold evaluation, and local alert generation.

The camera-processing pipeline runs on a computer, while the ESP32 processes sensor-side signals and supports local alert operation. The integrated design aims to provide useful crowd information and timely local warnings to support crowd management.

3.4 System Architecture Diagram

The system architecture of CrowdGuard AI describes the arrangement of the hardware and software components and explains how they interact to monitor crowd occupancy and generate alerts.

The proposed architecture consists of four major layers: Sensing Layer, Perception Layer, Decision-Making Layer, and Response Layer. Each layer performs a specific function in the crowd monitoring process.

The system uses a camera to capture video from the monitored area. The video is processed using OpenCV, YOLOv8n, and ByteTrack to detect and track people. The resulting information is used to estimate occupancy within a selected monitoring zone.

At the same time, an IR/laser break-beam sensor detects crossing events and sends signals to an ESP32 microcontroller. The camera-based occupancy estimate and sensor-event information are compared during decision-making.

The system evaluates occupancy against configured thresholds and considers changes in crowd count over time. When the configured critical condition is satisfied, the response layer activates the local buzzer and LED.

3.4.1 Overall System Architecture

The following diagram represents the main processing flow of CrowdGuard AI.

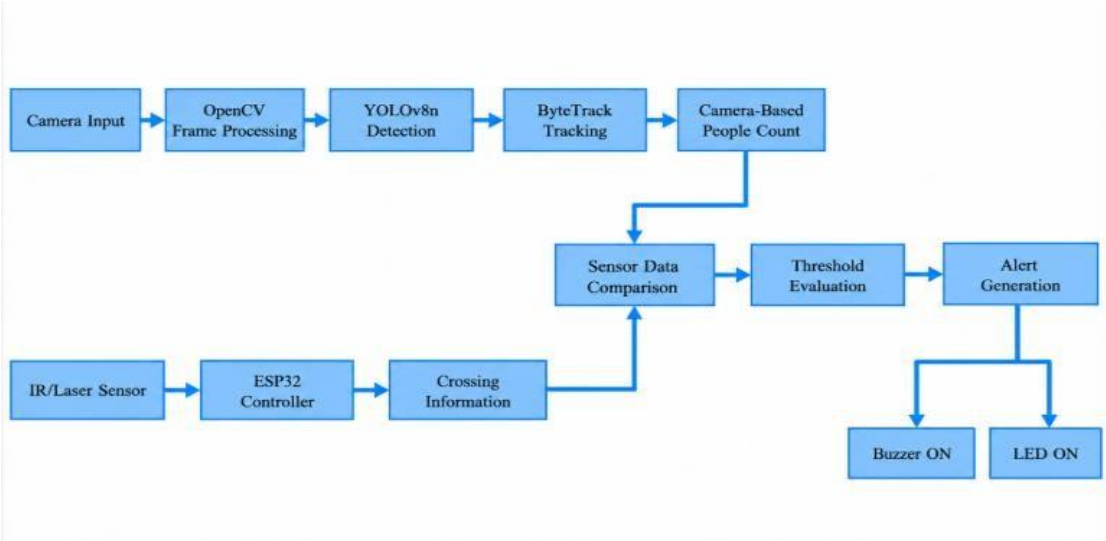


Figure 3.1: System Architecture of CrowdGuard AI

3.4.2 Sensing Layer

The sensing layer collects information from the physical environment.

It consists of the following components:

Camera: Captures video of the monitored area and provides the input for AI-based person detection.

IR/Laser Sensor: Detects interruptions in a light beam when a person or object crosses the monitored point.

The camera and sensor provide different types of information. The camera supports occupancy estimation across a selected region, while the beam sensor records crossing events at a specific location.

3.4.3 Perception Layer

The perception layer processes the camera input and extracts information about people.

It consists of three major components.

OpenCV: Reads the camera stream and processes individual video frames.

YOLOv8n: Detects people in the processed frames and generates bounding boxes around detected individuals.

ByteTrack: Associates detections across consecutive frames and maintains tracking information for detected people.

The output of this layer supports people counting and occupancy estimation within the selected monitoringzone.

3.4.4 Decision-Making Layer

The decision-making layer evaluates the information produced by the perception and sensing components.

Its major functions include:

1. Identifying the monitoring region.
2. Estimating the number of people in the selected region.
3. Calculating crowd density when the monitored area is known.
4. Comparing camera-based occupancy with sensor-event information.
5. Calculating changes in crowd count over time.
6. Evaluating occupancy against configured thresholds.
7. Determining whether an alert condition has been satisfied.

The camera count and sensor-event count are not simply added together because they may represent different measurements.

The system uses their relationship as a consistency-checking indicator and evaluates occupancy separately against the configured threshold.

3.4.5 Response Layer

The response layer communicates the monitoring result through local alert devices.

Buzzer: Produces an audible warning when the configured critical condition is satisfied.

LED: Provides a visual warning to indicate that an alert has been activated.

Monitoring Dashboard: Displays available monitoring information, such as the estimated people count, occupancy status, sensor-event count, processing speed, and other implemented metrics.

The response layer helps communicate important conditions to personnel responsible for monitoring the area.

3.4.6 Data Flow Between Components

The data flow between the components can be summarized as follows.

Table 3.3: Data Flow Between System Components

S. No.	Input Component	Processing Component	Output
1	Camera	OpenCV	Processed video frames
2	Processed frames	YOLOv8n	Person detections
3	Person detections	ByteTrack	Tracking information
4	Tracking information	ROI and counting logic	Camera-based occupancy estimate
5	IR/laser sensor	ESP32	Crossing-event information

S. No.	Input Component	Processing Component	Output
6	Camera estimate and sensor events	Sensor comparison logic	Consistency-checking information
7	Occupancy and monitoring parameters	Threshold evaluation	Monitoring status
8	Critical monitoring condition	Alert mechanism	Buzzer and LED activation

3.4.7 Advantages of the Architecture

The proposed architecture provides several advantages.

- **Modularity:** Each layer performs a specific function.
- **Integration:** AI-based video analysis and electronic sensing contribute to a common monitoring process.
- **Zone-Based Monitoring:** Occupancy can be evaluated within a selected region.
- **Automated Decision Support:** Threshold logic helps determine when warnings are required.
- **Local Response:** The buzzer and LED provide immediate local indications.
- **Expandability:** Additional zones and sensors may be incorporated in future versions.

Summary

The system architecture explains how CrowdGuard AI combines camera-based person detection, multi-object tracking, IR/laser sensing, threshold evaluation, and local alert generation.

The camera-processing pipeline runs on a computer, while the ESP32 handles sensor-side operations and supports local alert control. This architecture provides the foundation for the hardware and software implementation described in the following sections.

3.5 Hardware Requirements

The hardware requirements of CrowdGuard AI include the physical devices needed to capture video, detect crossing events, process sensor signals, and generate alerts. The proposed system combines camera-based monitoring with IoT-based sensing to support crowd occupancy monitoring in temple festivals.

The hardware components are selected to support the major functions of the system, including person detection, multi-object tracking, sensor integration, and local alert generation.

The camera provides video input for the AI-based monitoring application. The IR/laser break-beam sensor detects interruptions in its beam, while the ESP32 processes the sensor signals and supports the local alert mechanism. A computer or laptop runs the video-processing application and displays the monitoring information.

3.5.1 List of Hardware Components

The following table presents the main hardware components used in the CrowdGuard AI prototype.

Table 3.4: List of Hardware Components

S. No.	Hardware Component	Purpose
1	ESP32-CAM / Webcam	Captures video of the monitored area.
2	ESP32 Development Board	Processes sensor signals and supports local device control.
3	IR/Laser Break-Beam Sensor	Detects interruptions in the light beam.
4	Buzzer	Produces an audible warning when activated.
5	LED	Provides a visual warning indication.
6	Breadboard	Supports temporary circuit assembly during prototyping.
7	Jumper Wires	Connect the sensor, controller, and other electronic components.
8	Laptop / Computer	Runs the AI-based video-processing application and monitoring dashboard.
9	USB Cable	Provides the required connection for programming and communicating with the development board.
10	Power Supply	Provides suitable power to the electronic components.

Note: The exact component configuration may vary between development and final prototype arrangements. The computer runs the main AI video-processing pipeline in the current implementation.

3.5.2 Camera Module

The camera is responsible for capturing video of the monitored environment. The video stream provides the input required for person detection and tracking.

The project supports the use of an ESP32-CAM or webcam as the video input source. During development, a webcam can be used to test the software pipeline before integrating the intended camera hardware.

The captured video is processed by OpenCV, and the resulting frames are passed to YOLOv8n for person detection.

Functions:

- Captures video from the monitored area.
- Provides input for person detection.
- Supports tracking and occupancy estimation.
- Helps identify people within the selected monitoring zone.

3.5.3 ESP32 Development Board

The ESP32 is a microcontroller used to interface with electronic sensors and output devices.

In CrowdGuard AI, the ESP32 receives signals from the IR/laser sensor and processes them according to the programmed logic. It also supports the local alert mechanism.

Functions:

- Reads the sensor output.
- Processes crossing-event signals.
- Supports control of the buzzer and LED.
- Provides communication capabilities for integration with other system components.

The ESP32 handles sensor-side operations, while the computer performs the main AI-based video processing.

3.5.4 IR/Laser Break-Beam Sensor

The IR/laser break-beam sensor detects an interruption in a light beam between a source and a receiver.

When a person or object passes through the beam, the receiver's output changes. The ESP32 reads this signal and processes it as a crossing event.

Functions:

- Detects beam interruptions.
- Provides crossing-event information.
- Supplies an independent sensing input alongside camera-based monitoring.
- Supports sensor data comparison.

The sensor monitors a specific point and cannot independently determine the distribution of people throughout a larger area.

3.5.5 Buzzer

The buzzer is an electronic output device that produces an audible signal when activated.

In the proposed system, the buzzer provides a local warning when the required alert condition is satisfied.

Functions:

- Produces an audible warning.
- Communicates critical monitoring conditions.
- Helps attract the attention of nearby personnel.

The buzzer's activation depends on the system's programmed alert logic.

3.5.6 LED Indicator

The LED provides a visual indication when an alert is activated.

It operates as part of the local alert mechanism and complements the audible warning produced by the buzzer.

Functions:

- Provides a visible alert indication.
- Helps communicate the system's alert state.
- Complements the buzzer during critical conditions.

The LED and buzzer together provide two different forms of local notification.

3.5.7 Breadboard and Jumper Wires

A breadboard and jumper wires are used to assemble and connect the electronic components during prototyping.

The breadboard allows the circuit to be tested without requiring a permanent printed circuit board. Jumper wires connect the sensor outputs and electronic devices to the appropriate microcontroller pins.

Functions:

- Supports temporary circuit assembly.
- Connects the sensor to the ESP32.
- Simplifies testing and troubleshooting.
- Allows wiring changes during development.

The components must be connected according to their electrical requirements and the selected ESP32 board's pin configuration.

3.5.8 Laptop or Computer

The laptop or computer provides the main processing environment for CrowdGuard AI.

It runs the Python application and the required computer vision and AI libraries. The system processes video frames, detects people, tracks their movement, estimates occupancy, and evaluates monitoring conditions.

Functions:

- Runs Python and OpenCV.
- Executes the YOLOv8n detection model.
- Performs ByteTrack-based tracking.

- Estimates occupancy within the monitoring region.
- Displays monitoring information.
- Supports the decision-making and alert logic.

The computer is important because the current prototype performs the main video-processing operations outside the ESP32 microcontroller.

3.5.9 Hardware Requirements Summary

The hardware components work together to support the complete monitoring process.

Table 3.5: Hardware Requirements Summary

Component	Input / Output	Main Role
Camera	Video input	Captures the monitored scene.
ESP32-CAM / Webcam	Video source	Supplies camera frames to the monitoring application.
IR/Laser Sensor	Sensor signal	Detects crossing events.
ESP32 Development Board	Electronic input and output	Processes sensor signals and supports alert control.
Buzzer	Audible output	Generates a local warning.
LED	Visual output	Indicates the alert state.
Breadboard and Jumper Wires	Electrical connections	Support prototype assembly.
Laptop / Computer	Video processing and display	Runs the AI application and monitoring dashboard.
USB Cable and Power Supply	Connection and power	Support programming, communication, and device operation.

Summary

The hardware requirements of CrowdGuard AI consist of a camera, ESP32 development board, IR/laser break-beam sensor, buzzer, LED, supporting circuit components, and a computer.

These components provide the physical infrastructure required for camera-based monitoring, sensor-event detection, occupancy evaluation, and local alert generation.

3.6 Software Requirements

The software requirements of CrowdGuard AI include the programming languages, libraries, machine learning models, development tools, and supporting software required to implement the proposed crowd monitoring system.

The system uses artificial intelligence and computer vision to detect and track people in video frames. The resulting information is used to estimate crowd occupancy, evaluate configured thresholds, and generate alerts. The IoT components are programmed to process sensor signals and support the local alert mechanism.

Python is used for the main application, while OpenCV supports video processing and YOLOv8n performs person detection. ByteTrack is used for multi-object tracking. The ESP32 development board is programmed using a suitable microcontroller development environment.

3.6.1 List of Software Requirements

The following table presents the major software components used in the CrowdGuard AI prototype.

Table 3.6: List of Software Requirements

S. No.	Software / Technology	Purpose
1	Python 3.12	Main programming language for the monitoring application.
2	OpenCV 5.0.0.93	Captures and processes video frames.
3	Ultralytics 8.4.104	Provides the framework for running the YOLOv8n model.
4	YOLOv8n	Detects people in video frames.
5	ByteTrack	Tracks detected people across consecutive frames.
6	NumPy 2.4.6	Supports numerical and array operations.
7	PyTorch 2.13.0	Provides the deep-learning execution framework used by the model.
8	Torchvision 0.28.0	Provides supporting computer vision utilities.
9	Visual Studio Code	Used to write, edit, and debug Python code.
10	Arduino IDE 2.x	Used to develop and upload firmware to the ESP32.
11	Windows 10/11	Operating system used for development and execution.

Note: The versions listed above are the versions recorded for the project environment. Compatibility should be verified when recreating the environment on another computer.

3.6.2 Python Programming Language

Python is the primary programming language used to develop the CrowdGuard AI monitoring application.

Python provides libraries for computer vision, deep learning, numerical processing, and application development. Its extensive library support makes it suitable for integrating person detection, tracking, counting, and alert logic.

In the proposed system, Python performs the following functions:

- Reads video input from the camera.
- Processes video frames.
- Runs the YOLOv8n detection model.
- Performs ByteTrack-based tracking.
- Estimates occupancy within the selected monitoring region.
- Calculates crowd density and growth rate where required.
- Evaluates occupancy against configured thresholds.
- Supports sensor-data comparison and alert logic.

Python acts as the main software component connecting the AI processing stages.

3.6.3 OpenCV

OpenCV is a computer vision library used to process images and videos.

In CrowdGuard AI, OpenCV handles camera input and prepares video frames for analysis by the detection model. It also supports drawing bounding boxes and displaying monitoring information.

Functions of OpenCV

1. Captures video from the selected camera.
2. Reads individual video frames.
3. Processes frames before model inference.
4. Displays detected people and tracking information.
5. Supports region-of-interest selection and visualization.
6. Helps display monitoring results.

OpenCV provides the video-processing foundation for the proposed system.

3.6.4 YOLOv8n Object Detection Model

YOLOv8n is the lightweight nano variant of the YOLOv8 object detection model family [14].

The model is used to identify people in the incoming video frames. It generates bounding boxes and confidence scores for detected objects [1], [13].

In CrowdGuard AI, the person detections provide the input for the tracking and occupancy estimation stages [1], [2].

Functions

- Detects people in video frames.
- Identifies the approximate location of each detected person.
- Produces bounding boxes around detected individuals.
- Provides confidence scores for detections.
- Supports continuous video analysis.

The model's performance depends on image quality, lighting, occlusion, camera placement, and available processing resources [22], [29].

3.6.5 ByteTrack Tracking Algorithm

ByteTrack is a multi-object tracking algorithm used to associate detections across consecutive video frames [2].

A detection model identifies people in each frame, while a tracking algorithm helps maintain their identities over time [2], [16].

In CrowdGuard AI, ByteTrack uses detection information to associate people across frames [2], [15]. This tracking information supports occupancy estimation and reduces the likelihood of treating every frame-level detection as a completely new person [15].

Functions

1. Associates detections across consecutive frames.
2. Maintains track identities while people remain observable.
3. Supports movement monitoring.
4. Provides tracking information for zone-based counting.
5. Helps improve consistency in occupancy estimation.

Tracking can still be affected by occlusion, missed detections, and people entering or leaving the camera's field of view [15], [29].

3.6.6 NumPy

NumPy is a Python library used for numerical computation and array operations.

Computer vision and machine learning applications frequently use arrays to represent image data, coordinates, and numerical measurements.

In CrowdGuard AI, NumPy supports numerical operations associated with the monitoring pipeline.

Its functions may include:

- Handling numerical arrays.
- Performing mathematical calculations.
- Supporting coordinate and measurement operations.
- Processing numerical information used by the application.

3.6.7 PyTorch and Torchvision

PyTorch is a deep-learning framework used to execute neural network models [28]. It provides the computational operations required for model inference.

Torchvision provides supporting computer vision utilities and components that can be used alongside PyTorch [28].

Within CrowdGuard AI, these technologies support the execution of the YOLOv8n detection model through the installed software environment [14], [28].

Their performance depends on the selected model, available hardware, software compatibility, and execution configuration.

3.6.8 Visual Studio Code

Visual Studio Code is a source-code editor used for developing and maintaining the Python application.

It provides facilities for writing code, organizing project files, running programs, and identifying errors during development.

Functions

- Writing and editing Python programs.
- Organizing the project structure.
- Running the monitoring application.
- Debugging software errors.
- Managing the development workflow.

It provides a convenient environment for developing the computer vision and monitoring modules.

3.6.9 Arduino IDE

Arduino IDE is a development environment used to write and upload firmware to supported microcontroller boards.

In CrowdGuard AI, it can be used to program the ESP32 to read the IR/laser sensor output and control connected electronic devices.

Functions

1. Writing microcontroller firmware.
2. Configuring sensor input pins.
3. Reading digital sensor signals.
4. Processing crossing-event information.
5. Controlling the buzzer and LED.
6. Testing the electronic circuit during development.

The firmware must be configured according to the selected ESP32 board and its electrical connections.

3.6.10 Operating System

The project uses Windows 10/11 as its development and execution environment.

The operating system supports the installation and execution of Python, OpenCV, Ultralytics, PyTorch, Visual Studio Code, and the other required development tools.

It also provides the environment needed to connect the camera, execute the monitoring application, and communicate with supported development boards.

3.6.11 Software Integration

The software components work together to perform the crowd monitoring process.

The integration follows these stages:

1. Python starts the monitoring application.
2. OpenCV captures video from the camera.
3. YOLOv8n detects people in the video frames.
4. ByteTrack associates detections across frames.
5. The occupancy logic estimates the number of people in the selected region.
6. The application evaluates the crowd conditions against configured thresholds.
7. Sensor-event information from the ESP32 is incorporated into the monitoring decision according to the implemented communication and comparison logic.
8. The alert mechanism activates the local warning devices when the configured conditions are satisfied.

This integration connects the computer vision pipeline with the IoT sensor and alert components.

Summary

The software requirements of CrowdGuard AI include Python, OpenCV, YOLOv8n, ByteTrack, NumPy, PyTorch, Torchvision, Visual Studio Code, Arduino IDE, and Windows.

These technologies support video acquisition, person detection, multi-object tracking, occupancy estimation, sensor integration, and alert generation. Together, they form the software foundation of the proposed crowd monitoring system.

Table 3.7: Functional Requirements

S. No.	Functional Requirement	Description
1	Video Acquisition	Captures video from the selected camera.
2	Person Detection	Detects people using YOLOv8n.
3	Person Tracking	Associates detections using ByteTrack.
4	Zone Identification	Defines the region to be monitored.
5	People Counting	Estimates occupancy within the selected region.
6	Density Estimation	Calculates people per square metre when area information is available.
7	Sensor Event Detection	Detects beam interruptions using the IR/laser sensor.
8	Sensor Data Comparison	Compares camera-based occupancy with sensor-event information.
9	Growth-Rate Calculation	Estimates changes in crowd count over time.
10	Threshold Evaluation	Compares occupancy with configured limits.
11	Alert Generation	Activates the buzzer and LED when required.
12	Monitoring Display	Displays available crowd and system information.
13	Continuous Monitoring	Repeats the monitoring and evaluation process.

Table 3.8: Non-Functional Requirements

S. No.	Requirement	Description
1	Performance	Processes video and sensor information efficiently.
2	Reliability	Operates consistently under the tested conditions.
3	Accuracy	Provides useful person detection and occupancy estimates.
4	Real-Time Operation	Updates monitoring information and responds to configured conditions promptly.
5	Usability	Presents monitoring results in an understandable format.
6	Maintainability	Supports testing, troubleshooting, and software updates.
7	Scalability	Provides a foundation for adding monitoring zones and sensors.
8	Portability	Can be configured on compatible computing environments.
9	Safety	Supports responsible crowd monitoring and warning generation.
10	Compatibility	Ensures suitable integration between hardware and software components.

Table 3.9: Use Cases of the System

S. No.	Use Case	Description	Expected Result
1	Capture Video	The system obtains video from the selected camera.	Video frames become available for processing.
2	Detect People	YOLOv8n identifies people in the video frames.	Person detections are generated.
3	Track People	ByteTrack associates detections across frames.	Tracking information becomes available.
4	Estimate Occupancy	The system counts tracked people within the selected region.	An estimated people count is produced.
5	Detect Crossing Events	The IR/laser sensor detects beam interruptions.	Sensor-event information is recorded.
6	Compare Sensor Information	The system compares camera-based occupancy with sensor events.	Possible discrepancies can be identified.
7	Evaluate Threshold	The system checks occupancy against the configured threshold.	The monitoring status is determined.
8	Generate Alert	The configured critical condition activates the local alert mechanism.	The buzzer and LED provide a warning.
9	Observe Monitoring Status	The operator views the available monitoring information.	The operator can assess the displayed condition.

Table 3.10: Threshold Decision Examples

Current Occupancy	Threshold	System Decision
60 people	100 people	Continue monitoring
90 people	100 people	Continue monitoring
100 people	100 people	Threshold reached
120 people	100 people	Generate alert

Table 3.11: Warning Levels

Warning Level	Colour	Meaning	Suggested Action
Level 1	Green	Occupancy is within the normal operating range.	Continue monitoring.

Warning Level	Colour	Meaning	Suggested Action
Level 2	Yellow	Occupancy is approaching the configured threshold.	Observe the crowd condition.
Level 3	Orange	Occupancy requires increased attention.	Prepare for appropriate crowd-management action.
Level 4	Red	A critical configured alert condition has been reached.	Notify the responsible operator and take suitable action.

Table 3.12: Technology Summary

S. No.	Component	Technology Used	Purpose
1	Programming language	Python	Implements the AI-based monitoring logic.
2	Computer vision library	OpenCV	Captures and processes video frames.
3	Object detection model	YOLOv8n	Detects people in video frames.
4	Object tracking algorithm	ByteTrack	Tracks detected people across consecutive frames.
5	Numerical processing	NumPy	Supports numerical operations and data processing.
6	AI framework	PyTorch	Supports execution of the object detection model.
7	Microcontroller	ESP32	Processes signals from the connected sensor arrangement.
8	Camera	Webcam / ESP32-CAM	Provides video input for crowd monitoring.
9	IR/laser sensor arrangement	IR/laser break-beam sensor	Detects crossing events.
10	Alert devices	Buzzer and LED	Provide audible and visual alerts.
11	Development environment	Visual Studio Code and Arduino IDE	Support software development and microcontroller programming.

Table 3.13: Unit Test Cases

Test Case	Component	Testing Procedure	Expected Result
UT01	Camera input	Start the video stream.	Video frames are received.
UT02	OpenCV processing	Process the captured frames.	Frames are prepared for detection.
UT03	YOLOv8n detection	Provide frames containing people.	People are detected and bounding boxes are generated.
UT04	ByteTrack tracking	Process consecutive frames containing moving people.	Detected people are associated across frames when tracking conditions permit.
UT05	Occupancy estimation	Count tracked people in the monitoring zone.	An occupancy estimate is produced.
UT06	IR/laser sensor	Interrupt the sensor beam.	The sensor produces a corresponding signal.
UT07	ESP32 controller	Provide the sensor output to the controller.	The controller processes the sensor signal.

Test Case	Component	Testing Procedure	Expected Result

UT08	Threshold evaluation	Compare occupancy with the configured threshold.	The system determines whether the threshold condition is satisfied.
UT09	Buzzer	Trigger the configured alert output.	The buzzer activates.
UT10	LED	Trigger the configured visual alert output.	The LED activates.

Table 3.14: Camera-Based and IR/Laser Sensor Monitoring Compared

Parameter	Camera-Based Monitoring	IR/Laser Sensor Monitoring
Primary function	Estimates people present in the monitored area.	Detects crossing events at the sensor location.
Input	Video frames.	Sensor signals.
Processing component	YOLOv8n and ByteTrack.	ESP32 controller.
Output	Estimated occupancy.	Crossing-event information.
Main limitation	Detection may be affected by occlusion and lighting.	Does not independently measure total occupancy.

Table 3.15: Threshold Test Conditions

Test Condition	Estimated Occupancy	Configured Threshold	Expected System Behaviour
Low occupancy	10	50	Continue monitoring
Moderate occupancy	30	50	Continue monitoring
Threshold reached	50	50	Threshold reached
Threshold exceeded	60	50	Generate alert

Table 3.16: Performance Evaluation Methods

Component	Performance Aspect	Evaluation Method
Camera	Video acquisition	Check frame availability and continuity.
YOLOv8n	Person detection	Compare predictions with labelled video frames.
ByteTrack	Person tracking	Examine tracking continuity across frames.
Occupancy estimation	People counting	Compare estimated and manually verified counts.
IR/laser sensor	Crossing detection	Compare sensor events with controlled beam interruptions.
ESP32	Sensor signal processing	Verify that sensor signals are processed correctly.
Threshold module	Decision-making	Test occupancy values below, at, and above the threshold.
Buzzer and LED	Alert generation	Verify activation under the configured alert condition.

Table 3.17: Limitations of the Proposed System

S. No.	Limitation	Description
1	Detection limitations	Person detection may be affected by occlusion, lighting, and camera position.
2	Tracking limitations	Tracking continuity may decrease in highly crowded scenes.
3	Sensor coverage	The IR/laser sensor detects crossings at its sensing point rather than total zone occupancy.
4	Measurement differences	Camera-based occupancy and sensor event counts represent different measurements.
5	Hardware dependency	Processing speed depends on the available computer and hardware configuration.
6	Threshold dependency	Alert behaviour depends on suitable threshold settings.
7	Environmental conditions	Outdoor conditions may affect camera and sensor operation.
8	Limited validation	Broader testing is required to evaluate performance in large public gatherings.
9	Human supervision	Alerts support operators but do not independently manage crowd movement.
10	Remote notification	Mobile or remote alerts require additional implementation if they are not already integrated.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents the experimental results and performance analysis of the proposed CrowdGuard AI system. The system combines artificial intelligence, computer vision, IoT-based sensing, threshold evaluation, and local alert generation to support crowd monitoring in public gathering areas.

The evaluation focuses on person detection, crowd occupancy estimation, sensor operation, processing speed, crowd density monitoring, and alert response time. The experimental results are based on the prototype demonstration and additional evaluation outputs reported in the project paper.

The proposed system uses YOLOv8n for person detection and ByteTrack for tracking detected people across consecutive video frames. OpenCV processes the video input, while the ESP32 controller processes signals from the IR/laser sensor arrangement. The system evaluates occupancy against the configured threshold and activates the buzzer and LED when the alert condition is satisfied.

The results presented in this chapter describe the behaviour observed under the tested conditions. They should not be interpreted as a guarantee of performance in every real-world crowd environment.

4.2 Experimental Setup

The CrowdGuard AI prototype was evaluated using a computer-based video processing system and an IoT hardware arrangement. The computer handled AI-based detection and tracking, while the ESP32 controller processed the sensor signals.

The setup included a webcam or ESP32-CAM for video input, an ESP32 controller, an IR/laser sensor arrangement, a buzzer, and an LED. The demonstration was conducted using a classroom-sized monitoring area.

4.2.1 Hardware Setup

Table 4.1: Hardware Setup

S. No.	Hardware Component	Purpose
1	Computer or laptop	Runs the AI-based monitoring application.
2	Webcam / ESP32-CAM	Captures video of the monitored area.
3	ESP32 controller	Processes sensor signals and supports hardware control.
4	IR/laser sensor arrangement	Detects beam interruptions caused by crossing events.
5	Buzzer	Provides an audible alert.
6	LED	Provides a visual alert.
7	Breadboard and jumper wires	Support the hardware connections.

4.2.2 Software Setup

Table 4.2: Software Setup

S. No.	Software / Library	Purpose
1	Python	Implements the monitoring logic.
2	OpenCV	Captures and processes video frames.
3	YOLOv8n	Detects people in video frames.
4	ByteTrack	Tracks detected people across frames.
5	PyTorch	Supports execution of the detection model.
6	NumPy	Supports numerical processing.
7	Arduino IDE	Supports ESP32 programming.
8	Visual Studio Code	Supports software development.

The software and hardware components work together to implement the monitoring pipeline, from video acquisition and sensor processing to threshold evaluation and local alert generation.

4.3 Functional Demonstration Results

The functional demonstration was conducted in a monitored area of approximately 100 m^2. One visible person was detected, the crowd density was calculated, and the sensor and alert components were evaluated.

The reported observations are summarized below.

Table 4.3: Functional Demonstration Results

Parameter	Observed Result	Description
Test area	100 m²	Area used for the demonstration.
Visible people detected	1	One person was detected in the test scene.
Configured threshold	1 person	Threshold used in the functional demonstration.
Calculated crowd density	0.010 persons/m²	Density calculated from one person in 100 m².
Laser sensor events	4	Four sensor events were recorded.
Processing speed	18–25 FPS	Observed processing-speed range.
Dashboard frame rate	23 FPS	Frame rate displayed during the demonstration.
Tracking algorithm	ByteTrack	Enabled during monitoring.

The results indicate that the prototype demonstrated the integration of camera-based detection, occupancy calculation, sensor-event recording, and alert-related functions.

Important observation: The detection of one person in this demonstration verifies the system's operation for that particular test scene. It does not establish detection accuracy for large or densely packed crowds.

4.4 Person Detection and Tracking Results

The person detection module uses YOLOv8n to identify people in the video stream [1]. The detected bounding boxes are passed to ByteTrack, which associates detections across consecutive frames [2].

The evaluation reported the following metrics for the tested setup:

Table 4.4: Person Detection Evaluation Metrics

Evaluation Metric	Reported Result
Accuracy	0.92
Precision	0.89
Recall	0.93
F1-score	0.91
Mean Absolute Error (MAE)	0.05

These are the values reported in the project paper's evaluation. Their interpretation depends on the evaluation dataset, annotation procedure, and metric definitions used in the experiments.

The results suggest that the evaluation recorded both correct detections and errors. Precision measures the correctness of positive predictions, while recall measures how effectively actual positive cases were identified. The F1-score combines precision and recall into a single measure.

The MAE represents the average absolute difference between evaluated values and their corresponding reference values. Its interpretation depends on the particular quantity being evaluated.

ByteTrack supports tracking continuity across successive frames, which is important when estimating occupancy without repeatedly counting the same person in every frame [2], [15].

Further testing with more people, different crowd sizes, and varied lighting conditions is required to establish how consistently these results can be reproduced.

4.5 Crowd Density Calculation Results

Crowd density is calculated by dividing the estimated number of people by the area of the monitored zone.

The formula is:

D=CAD=\frac{C}{A}

Where:

- DD = Crowd density.
- CC = Number of people detected.
- AA = Area of the monitored zone in square metres.

For the functional demonstration:

C=1,A=100 m2C=1,\qquad A=100\text{ m}^2

Therefore,

D=1100D=\frac{1}{100} D=0.010 persons/m2\boxed{D=0.010\text{ persons/m}^2}

The calculated density matches the value reported for the demonstration.

The calculation illustrates how occupancy can be represented relative to the size of the monitored area. When the estimated number of people changes, the density can be recalculated to reflect the updated occupancy.

4.6 Sensor Validation Results

The IR/laser sensor arrangement was evaluated to check whether beam interruptions were recorded by the hardware setup.

During the reported demonstration, the sensor recorded four events. These events were processed as additional sensor information for the decision-making component.

The observations are summarized below.

Table 4.5: Sensor Validation Results

Test Parameter	Result	Interpretation
Sensor beam interruption	Detected	The sensor path produced detectable events.

Test Parameter	Result	Interpretation
Events recorded	4	Four laser sensor events were reported.
ESP32 involvement	Integrated	The controller processed sensor information.
Sensor fusion	Integrated	Camera and sensor information were used in the monitoring pipeline.

The sensor results demonstrate the recording of crossing events under the tested conditions.

The camera count and sensor event count must not be treated as identical quantities. The camera estimates occupancy within a region, whereas the sensor records crossing events at a specific point. Therefore, additional controlled crossing tests are required to evaluate sensor sensitivity, false triggers, and consistency between the two sources.

4.7 Processing Speed Analysis

Processing speed is an important factor in a real-time crowd monitoring system because video frames must be processed frequently enough to update the monitoring results.

The reported processing speed ranged from 18 to 25 frames per second (FPS) [5]. A frame rate of 23 FPS was displayed in the dashboard during the demonstration [1], [5].

Table 4.6: Processing Speed Observations

Parameter	Result
Minimum reported frame rate	18 FPS
Maximum reported frame rate	25 FPS
Dashboard frame rate	23 FPS

The results indicate that the prototype processed video at a measurable frame rate during the tested run.

However, the actual frame rate may change depending on hardware capability, video resolution, model settings, and the complexity of the scene [5]. Processing speed alone does not establish detection accuracy or the total time required to generate an alert [5], [23].

4.8 Crowd Growth and Trend Monitoring Results

Crowd growth rate describes how quickly the number of people changes over a given time interval. It provides additional information beyond the current occupancy value

G=Ct−Ct−ΔtΔtG=\frac{C_t-C_{t-\Delta

t}}{\Delta t} Where:

- GG = Crowd growth rate.
- CtC_t = Current crowd count.
- Ct−ΔtC_{t-\Delta t} = Previous crowd count.
- Δt\Delta t = Time interval.

The project paper reports a density-rate value of 0.574 per minute on the dashboard during the functional demonstration. This is a reported dashboard observation and should not be treated as a universal value or a general performance guarantee.

Monitoring changes over time allows the system to observe whether occupancy is increasing or decreasing. This information can support early-warning logic when interpreted alongside the current occupancy and the configured threshold.

Further evaluation using repeated measurements and different crowd sizes is necessary to determine the consistency and predictive value of the trend-monitoring function.

4.9 Alert Generation Performance

The alert generation component was evaluated to determine whether the system activated the local warning mechanism when the configured critical condition was reached.

The project paper reports successful functional operation of the alert mechanism, including activation of the buzzer and recording of the critical alert condition. It also reports response-time measurements for three configurations.

Table 4.7: Alert Response Time by Configuration

Configuration	Reported Response Time
Camera-based configuration	1.8 seconds
Laser-sensor configuration	1.2 seconds
Combined sensor-fusion configuration	0.6 seconds

The reported measurements show that the combined camera–laser sensor-fusion configuration had the lowest response time among the three evaluated configurations.

These values apply to the test conditions and measurement procedure described in the project paper. Repeated tests are necessary to assess their variability and confirm whether the same response-time differences can be reproduced.

The local alert mechanism supports operator awareness by providing audible and visual warnings. Remote or mobile notifications remain a separate extension unless implemented and tested independently.

4.10 Overall Results and Discussion

The experimental observations indicate that the CrowdGuard AI prototype integrates person detection, tracking, occupancy calculation, sensor-event recording, threshold evaluation, and local alert generation.

The major findings are summarized below:

1. **Person detection:** The prototype detected one visible person during the functional demonstration.
2. **Occupancy calculation:** The system calculated a density of 0.010 persons/m² for one person in a 100 m² area.
3. **Sensor operation:** Four laser sensor events were recorded during the reported demonstration.
4. **Processing performance:** The observed frame rate ranged from 18 to 25 FPS.
5. **Trend monitoring:** A density-rate value of 0.574 per minute was displayed during the demonstration.
6. **Alert response:** The reported response times were 1.8 seconds for camera-based monitoring, 1.2 seconds for the laser-sensor configuration, and 0.6 seconds for the combined configuration.
7. **System integration:** The demonstration linked the perception, sensing, decision, and local-alert components into one monitoring workflow.

These observations provide evidence of prototype functionality under the tested conditions. However, the single-person demonstration and the additional evaluation metrics represent different forms of evidence and should not be combined as though they came from one identical test.

A larger validation study involving several people, different occupancy levels, multiple zones, varying lighting, and repeated trials is required to assess general performance and suitability for large public gatherings.

4.11 Chapter Summary

This chapter presented the experimental setup, functional demonstration results, person detection and tracking metrics, crowd density calculation, sensor validation, processing speed, trend monitoring, and alert response performance of CrowdGuard AI.

The reported results demonstrate the integration of computer vision, IoT-based sensing, threshold evaluation, and local alert generation in the prototype. The system detected a person in the functional demonstration, recorded sensor events, calculated crowd density, and operated at a reported frame rate of 18–25 FPS.

The project paper also reported evaluation metrics and response-time measurements from additional tests. These results are specific to the tested setup and require further validation before they can be generalized to large public gatherings.

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Introduction

Crowd safety is an important concern during temple festivals, religious processions, and other large public gatherings [6], [8]. A sudden increase in crowd occupancy can create congestion and increase the risk of crowd-related incidents [8]. Therefore, continuous monitoring and timely identification of potentially unsafe conditions are important.

The CrowdGuard AI project proposes an intelligent crowd monitoring system that combines artificial intelligence, computer vision, IoT-based sensing, and automated local alerts. The system uses YOLOv8n for person detection, ByteTrack for object tracking, and an IR/laser sensor arrangement connected to an ESP32 controller to obtain additional crossing information.

The proposed system estimates crowd occupancy, calculates crowd density, evaluates configured thresholds, and activates a buzzer and LED when the alert conditions are satisfied. It is intended to assist operators in monitoring crowd conditions and making timely decisions.

This chapter summarizes the project, discusses its major contributions, identifies its limitations, and presents possible future enhancements.

5.2 Conclusion

The CrowdGuard AI project presents an integrated approach to intelligent crowd monitoring. It combines camera-based person detection and tracking with IR/laser sensor information to support occupancy monitoring and threshold-based alert generation.

The system uses YOLOv8n to detect people in video frames and ByteTrack to track detected people across consecutive frames [1], [2]. OpenCV supports video capture and processing, while the ESP32 controller processes signals from the IR/laser sensor arrangement.

The estimated number of people in the monitored zone is used to calculate crowd density and evaluate occupancy against a predefined threshold. Sensor information provides additional crossing-event data that can be compared with camera-based measurements. When the configured alert condition is satisfied, the system activates the buzzer and LED to provide audible and visual warnings.

The prototype demonstration was conducted in a classroom-sized area of approximately 100 m^2. The reported demonstration detected one visible person, calculated a crowd density of 0.010 persons/m^2, recorded four laser sensor events, and operated at a processing speed of approximately 18–25 FPS. Additional evaluation outputs reported detection metrics and response-time measurements under the tested conditions.

These results demonstrate the integration of the main monitoring components in the prototype. However, further testing with larger crowds, multiple monitoring zones, and different environmental conditions is necessary before drawing conclusions about performance in large public gatherings.

In conclusion, CrowdGuard AI provides a foundation for combining computer vision, IoT sensing, occupancy analysis, and local alerts into a unified crowd monitoring system. It can support human operators by providing information about crowd conditions and identifying situations that may require attention.

5.3 Major Contributions of the Project

The major contributions of CrowdGuard AI are described below.

5.3.1 AI-Based Person Detection

The system uses YOLOv8n to detect people in video frames [1], [5]. This allows the monitoring process to identify people automatically rather than relying entirely on manual observation.

5.3.2 Multi-Object Tracking

ByteTrack is used to associate detected people across consecutive frames [2]. This supports continuous tracking and helps reduce repeated counting when estimating occupancy.

5.3.3 Zone-Based Occupancy Monitoring

The system estimates the number of people within a selected monitoring zone. This allows crowd conditions to be evaluated for a specific area of interest.

5.3.4 IoT-Based Sensor Integration

The project integrates an IR/laser sensor arrangement with an ESP32 controller [19], [21]. The sensor provides crossing-event information that complements the camera-based monitoring process [19], [21].

5.3.5 Sensor Data Comparison

The system compares camera-based occupancy estimates with sensor-event information [20]. This supports consistency checking between the two sources while recognizing that they measure different quantities.

5.3.6 Crowd Density and Growth Analysis

The system uses mathematical calculations to estimate crowd density and observe changes in occupancy over time. These measurements can support the identification of increasing crowd levels.

5.3.7 Local Alert Generation

The system uses a buzzer and LED to provide audible and visual warnings when the configured alert conditions are satisfied. These warnings help notify operators of conditions requiring attention.

5.4 Limitations of the Project

Although the proposed system demonstrates the integration of AI-based monitoring and IoT sensing, several limitations remain.

1. Detection limitations: Person detection may be affected by poor lighting, overlapping people, and partial visibility [22], [29].

2. Tracking limitations: ByteTrack may experience tracking interruptions when people become heavily occluded or disappear from the camera view [2], [15].

3. Sensor limitations: The IR/laser sensor detects crossing events at a specific point and cannot independently determine total crowd occupancy [20].

4. Measurement differences: Camera-based occupancy and sensor-event counts may differ because they represent different measurements.

5. Processing dependency: Real-time performance depends on the processing capability of the computer and the configuration of the AI model.

6. Environmental limitations: Outdoor conditions such as changing illumination, rain, and dust may affect camera and sensor performance.

7. Limited experimental validation: The initial functional demonstration used a controlled classroom-sized area. Larger-scale testing is needed to assess the system under different crowd conditions.

8. Human supervision: The system provides alerts and monitoring information but does not independently manage crowd movement or guarantee the prevention of crowd-related incidents.

These limitations indicate that additional testing and development are required before deployment in demanding real-world environments.

5.5 Future Enhancements

The CrowdGuard AI system can be improved through several future developments.

5.5.1 Multi-Zone Crowd Monitoring

Future versions can support multiple monitoring zones with separate occupancy estimates and configurable thresholds. This would help operators identify areas where crowd conditions require attention.

5.5.2 Improved Sensor Fusion

The sensor fusion mechanism can be enhanced by aligning camera measurements and sensor events over appropriate time intervals [20]. Improved decision rules could help identify inconsistencies and handle unreliable sensor readings [20].

5.5.3 Advanced Crowd Growth Prediction

Future development can incorporate more extensive time-series analysis to estimate changes in occupancy and identify rapidly increasing crowd levels [8].

The system could evaluate occupancy, growth rate, and zone-specific thresholds together to support earlier warnings. The reliability of such predictions would need to be established through additional experiments.

5.5.4 Remote Monitoring and Notifications

A future version could provide a secure remote dashboard or mobile notification system for authorized operators.

Such a system could communicate information about the monitored zone, occupancy, warning level, and alert status. Remote notifications would require additional implementation and testing for communication reliability, security, and response time.

5.5.5 Improved Camera Coverage

Multiple cameras could be installed at different locations to monitor larger areas, including entrances, exits, queue corridors, and narrow pathways.

This would provide broader visual coverage, although overlapping camera views and duplicate counting would need to be handled carefully.

5.5.6 Enhanced Environmental Performance

Future development could evaluate improved camera placement, hardware protection, and suitable sensing arrangements for outdoor environments.

Testing under different lighting conditions, weather conditions, and crowd sizes would help identify the changes required for more reliable operation.

5.5.7 Extended Experimental Validation

Further experiments should include:

- Different crowd sizes and occupancy levels.
- Multiple monitoring zones.
- Repeated sensor crossing tests.
- Videos containing occlusion and varying lighting.
- Counting accuracy against manually verified reference counts.

- Repeated measurements of processing speed and alert response time.
- Long-duration operation to evaluate hardware and communication reliability.

These experiments would provide stronger evidence of the system's reliability and practical usefulness.

5.6 Applications of CrowdGuard AI

The proposed system may be useful in several environments where crowd occupancy requires continuous observation.

Table 5.1: Application Areas of CrowdGuard AI

S. No.	Application Area	Possible Use
1	Temples	Monitoring queues and gathering areas.
2	Religious festivals	Observing crowd occupancy during events.
3	Public processions	Monitoring selected routes and gathering points.
4	Railway stations	Observing passenger congestion in selected zones.
5	Bus terminals	Monitoring waiting areas and entry points.
6	Exhibition centres	Observing occupancy in designated areas.
7	Public event venues	Supporting crowd monitoring during organized events.

These are potential application areas. Suitability for each environment must be established through location-specific testing and appropriate threshold configuration.

5.7 Final Conclusion

CrowdGuard AI demonstrates a combined approach to crowd monitoring through artificial intelligence, computer vision, IoT-based sensing, and local alert generation.

The use of YOLOv8n and ByteTrack supports person detection and tracking, while the IR/laser sensor arrangement provides additional crossing-event information. Occupancy estimation, crowd density calculations, and threshold evaluation help determine when an alert is required. The buzzer and LED provide local warnings to assist operators.

The prototype results demonstrate the integration of the main system components under the tested conditions. However, broader validation is necessary to establish reliability in large and complex public gathering environments.

With further improvements in sensor fusion, multi-zone monitoring, trend analysis, and remote monitoring, the system could be developed into a more comprehensive crowd monitoring solution.

Overall, CrowdGuard AI provides a foundation for intelligent crowd monitoring that aims to support timely awarenes informed decision-making and improved crowd management practices.

REFERENCES

The following references are taken from the reference list in your CrowdGuard AI research paper. Use the same reference numbering throughout your report where applicable.

1. Z. Chen, X. Xie, T. Qiu, and L. Yao, "Dense-Stream YOLOv8n: A Lightweight Framework for Real-Time Crowd Monitoring in Smart Libraries," *Scientific Reports*, vol. 15, Art. no. 11618, 2025, doi: 10.1038/s41598-025-94659-x.
2. Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in *Computer Vision – ECCV 2022*, pp. 1–21, 2022, doi: 10.1007/978-3-031-20047-2_1.
3. Y. Li, X. Zhang, and D. Chen, "CSRNet: Dilated Convolutional Neural Networks for Understanding the Highly Congested Scenes," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 1091–1100, doi: 10.1109/CVPR.2018.00120.
4. B. Pardamean, F. Abid, T. W. Cenggoro, G. N. Elwirehardja, and H. H. Muljo, "Counting People Inside a Region-of-Interest in CCTV Footage with Deep Learning," *PeerJ Computer Science*, vol. 8, e1067, 2022, doi: 10.7717/peerj-cs.1067.
5. H. H. Nguyen, V. T. Bui, T. N. Ta, H. M. Pham, C. N. Nguyen, and D. M. Nguyen, "YOLO Based Real-Time Human Detection for Smart Video Surveillance at the Edge," in *Proceedings of the IEEE International Conference on Communications and Electronics (ICCE)*, 2021, pp. 439–444, doi: 10.1109/ICCE48956.2021.9352144.
6. J. Reddy et al., "Crowd Control and Monitoring Using Deep Learning," in *Proceedings of the 2024 Second International Conference on Networks, Multimedia and Information Technology (NMITCON)*, 2024, doi: 10.1109/NMITCON62075.2024.10698968.
7. S. Wang, Z. Pu, Q. Li, and Y. Wang, "Estimating Crowd Density with Edge Intelligence Based on Lightweight Convolutional Neural Networks," *Expert Systems with Applications*, vol. 206, Art. no. 117823, 2022, doi: 10.1016/j.eswa.2022.117823.
8. A. C. Cob-Parro et al., "Real-Time Stampede Detection Using Optical Flow and Machine Learning Techniques," *IEEE Access*, vol. 9, 2021.
9. L. Boominathan, S. S. S. Kruthiventi, and R. Venkatesh Babu, "CrowdNet: A Deep Convolutional Network for Dense Crowd Counting," in *Proceedings of the ACM International Conference on Multimedia*, 2016.
10. Y. Zhang, D. Zhou, S. Chen, S. Gao, and Y. Ma, "Single-Image Crowd Counting via Multi-Column Convolutional Neural Network," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 589–597, doi: 10.1109/CVPR.2016.70.
11. H. Idrees, I. Saleemi, C. Seibert, and M. Shah, "Multi-Source Multi-Scale Counting in Extremely Dense Crowd Images," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013, pp. 2547–2554, doi: 10.1109/CVPR.2013.329.

12. V. A. Sindagi and V. M. Patel, “A Survey of Recent Advances in CNN-Based Single Image Crowd Counting and Density Estimation,” *Pattern Recognition Letters*, vol. 107, pp. 3–16, 2018, doi: 10.1016/j.patrec.2017.07.007.
13. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.
14. J. Terven, D.-M. Cordova-Esparza, and J.-A. Romero-Gonzalez, “A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS,” *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023, doi: 10.3390/make5040083.
15. A. Milan et al., “MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking,” *International Journal of Computer Vision*, vol. 129, pp. 834–849, 2021, doi: 10.1007/s11263-020-01393-0.
16. N. Wojke, A. Bewley, and D. Paulus, “Simple Online and Realtime Tracking with a Deep Association Metric,” in *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2017, pp. 3645–3649, doi: 10.1109/ICIP.2017.8296962.
17. S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 91–99.
18. T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in *Computer Vision – ECCV 2014*, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.
19. D. R. Arun, C. C. Columbus, A. Bhuvanesh, and A. S. Sumithra, “Smart Crowd Monitoring System Using IoT-Based YOLO-GHOST,” *Revue Roumaine des Sciences Techniques – Série Électrotechnique et Énergétique*, vol. 69, no. 3, pp. 341–346, 2024, doi: 10.59277/RRST-EE.2024.69.3.15.
20. A. M. Al-Shaery et al., “Crowd Management Challenges: A Tackling Approach Using Multi-Sensor Fusion,” *IEEE Access*, 2019.
21. F. Aldosari, S. Alamri, R. Alharbi, and G. Alghamdi, “IoT Based Crowd Management Tool,” in *Proceedings of the SAI International Systems Conference*, 2022, pp. 533–539.
22. N. Thakur, P. Nagrath, R. Jain, D. Saini, N. Sharma, and D. J. Hemanth, “Autonomous Pedestrian Detection for Crowd Surveillance Using Deep Learning Framework,” *Soft Computing*, vol. 27, no. 14, pp. 9383–9399, 2023.
23. M. Ş. Gündüz and G. Işık, “A New YOLO-Based Method for Real-Time Crowd Detection from Video and Performance Analysis of YOLO Models,” *Journal of Real-Time Image Processing*, vol. 20, 2023.
24. L. Boominathan, S. S. S. Kruthiventi, and R. Venkatesh Babu, “CrowdNet: A Deep Convolutional Network for Dense Crowd Counting,” in *Proceedings of the ACM Multimedia Conference*, 2016, pp. 640–644.
25. C. Wang, H. Zhang, L. Yang, S. Liu, and X. Cao, “Deep People Counting in Extremely Dense Crowds,” in *Proceedings of the 23rd ACM International Conference on Multimedia*, 2015, pp. 1299–1302.
26. C. Zhang, H. Li, X. Wang, and X. Yang, “Cross-Scene Crowd Counting via Deep Convolutional Neural Networks,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 833–841.

27. K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
28. A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8026–8037.
29. S. Shao, Z. Zhao, B. Li, T. Xiao, G. Yu, X. Zhang, and J. Sun, “CrowdHuman: A Benchmark for Detecting Human in a Crowd,” *arXiv preprint arXiv:1805.00123*, 2018.
30. P. Thanasutives, K.-I. Fukui, M. Numao, and C. Kijsirikul, “Encoder-Decoder Based Convolutional Neural Networks with Multi-Scale-Aware Modules for Crowd Counting,” *arXiv preprint arXiv:2003.05586*, 2020.

APPENDIX A: SOURCE CODE

A.1 Imports, ESP32 Serial Link and Model Loading

The program starts by importing OpenCV, NumPy, the Windows beep function, the serial library and the Ultralytics YOLO class. It opens the serial port COM5 at 115200 baud totalk to the ESP32, imports the zone-selection helpers from utils/zone_selector.py,

```
import cv2
import numpy as np
import winsound
import serial
esp32 = serial.Serial("COM5", 115200, timeout=1)
beam_status = 0
beam_count = 0
print("ESP32 connected on COM5")
from ultralytics import YOLO
from datetime import datetime
import time
import threading
from utils.zone_selector import
    ( mouse_callback,
    draw_zone,
    get_zone,
    reset_zone
)

# ====================================
# Load YOLO Model
# ====================================
print("Loading YOLOv8n...")
model = YOLO("yolov8n.pt")
print("Model Loaded Successfully!")
```

and loads the YOLOv8n weights.

Camera Input, Constants and Alarm Function

This part opens the camera stream, creates the CrowdGuard AI window and registers the mouse callback used to draw the monitoring zone. It also defines the dashboard colours, the crowd threshold and the play_alarm function, which beeps repeatedly on a separate thread while alarm_active is True, and initialises the FPS variables.

```
# ====================================
# Open Webcam
# ====================================
cap = cv2.VideoCapture("http://192.168.0.5:81/stream")
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
cv2.namedWindow("CrowdGuard AI")
cv2.setMouseCallback("CrowdGuard AI", mouse_callback)

if not cap.isOpened():
    print("Unable to open webcam")
    exit()

# ====================================
# Colors
# ====================================
BLACK = (20, 20, 20)
DARK = (35, 35, 35)
WHITE = (255, 255, 255)
GREEN = (0, 255, 100)
YELLOW = (0, 220, 255)
RED = (0, 0, 255)
BLUE = (255, 120, 0)
GRAY = (160, 160, 160)

font = cv2.FONT_HERSHEY_SIMPLEX
# ============================================================
# CROWD THRESHOLD
# ============================================================

CROWD_THRESHOLD = 1
```

```
# ============================================================

def play_alarm():

    while alarm_active:

        winsound.Beep(1200, 500)

        time.sleep(0.3)
# ============================================================
# FPS VARIABLES
# ============================================================

prev_time = time.time()
fps = 0
```

A.2 Main Loop: Serial Input and Frame Capture

Each pass of the main loop reads any pending line from the ESP32 and, for a BEAM message, updates beam_status and beam_count. It then reads a camera frame, resizes it to 960 x 720 pixels and places it on a 1280 x 720 dashboard canvas, leaving the right-hand side free for the information panel.

```
## ============================================================
# MAIN LOOP
# ============================================================

while True:

    if esp32.in_waiting:
        data = esp32.readline().decode(errors="ignore").strip()
        print("ESP32 DATA:", data)

        if data.startswith("BEAM,"):
            parts = data.split(",")

            if len(parts) == 3:
                try:
                    beam_status = int(parts[1])
                    beam_count = int(parts[2])
                except ValueError:
                    pass

    # ============================================================
    # READ FRAME
    # ============================================================

    # ============================================================
    # READ FRAME
    # ============================================================

        ret, frame = cap.read()

        if not ret:
            break
        frame = cv2.resize(frame, (960, 720))

        dashboard = np.zeros((720, 1280, 3), dtype=np.uint8)
        dashboard[:, :960] = frame
        frame = dashboard
```

A.3 Monitoring Zone, Person Detection and Zone Counting

The selected zone is drawn on the frame and the classroom is fixed at 10 x 10 (100 m2). YOLOv8n is run on the frame with a confidence of 0.5 and an image size of 320. For every person detection the program draws the bounding box, the ID label and the centre point, and uses cv2.pointPolygonTest to count the person in zone_people when the centre lies inside the zone.

```
    # ============================================================
    # DRAW ZONE
    # ============================================================

draw_zone(frame)
    # ============================================================
    # MONITORING ZONE
    # ============================================================

zone = get_zone()

    # Fixed classroom dimensions
CLASSROOM_LENGTH = 10
CLASSROOM_WIDTH = 10

    # Fixed classroom area
CLASSROOM_AREA = CLASSROOM_LENGTH * CLASSROOM_WIDTH
PEOPLE_PER_SQM = 1
CROWD_THRESHOLD = 1

    # Convert selected zone points into NumPy array
if len(zone) >= 3:

        zone_array =
            np.array( zone,
            dtype=np.int32
        )
    # ============================================================
    # INSTRUCTIONS
    # ============================================================

cv2.putText(
        frame,
        "Left Click: Add Point | R: Reset Zone | Q: Quit",
        (20, 90),
        font,
        0.7,
        YELLOW,
        2
    )

    # ============================================================
    # YOLO DETECTION
    # ============================================================
results =
    model.predict( frame
    ,
    conf=0.5,
    imgsz=320,
    verbose=False
)

    # ============================================================
    # PEOPLE COUNT
    # ============================================================

total_people = 0
zone_people = 0

    # ============================================================
    # PROCESS YOLO RESULTS
    # ============================================================

for result in results:

        if result.boxes is None:
            continue

        for box in result.boxes:

          ------------------------------------------------------------------
```

APPENDIX B

SCREENSHOT

